



The SemOps Manual

Running a semantic practice the way you run software

Semantechs

2026-08-13

Contents

Introduction	3
What this is	3
The shape of the argument	4
Read it on paper	5
How to read it	5
Conventions	5
1. The business case	6
1.1 The four business problems that are actually semantic	6
1.2 Why the payoff is invisible	8
1.3 The decay problem, which is the SemOps problem	8
1.4 What to write on the slide	9
Sources	9
2. People and cognition	10
2.1 Definitions are territory	10
2.2 The priesthood problem	10
2.3 Cognitive styles: people do not categorise the same way	11
2.4 Cognitive capacity is a hard budget	11
2.5 The workshop is a social event with a technical output	12
2.6 What this implies for the toolchain	12
Sources	13
3. Across the boundary: peers and supply chains	14
3.1 What changes at the boundary	14
3.2 Three topologies, three different problems	15
3.3 The cross-boundary release problem	17
3.4 Trust, provenance and attribution	19
3.5 Where this leaves the practice	19
Sources	19
4. From research to industry	20
4.1 Different objective functions	20
4.2 What actually arrives	20
4.3 A verified example from this manual's own toolchain	21

4.4 The evaluation mismatch	22
4.5 How to consume research output well	22
4.6 The other direction	23
Sources	23
5. GenAI and agents	24
5.1 AI for semantics: generation gets cheap	24
5.2 Semantics for AI: the grounding direction	25
5.3 Agents as participants in the pipeline	25
5.4 What does not change	27
5.5 The practical position	27
Sources	27
6. Stages and stories	28
6.1 The nine stages	28
6.2 The six layers, and who is accountable	29
6.3 The five maturity levels	29
6.4 The stories	32
6.5 The whole frame, once	33
7. The toolchain	34
7.1 The idea: one registry, two runtimes	34
7.2 Installing them	35
7.3 Decision table: question → command	35
7.4 Two flags worth knowing before you start	36
7.5 The fixture used throughout Part III	36
7.6 Where the deeper documentation lives	37
8. Model and validate	38
8.1 The inner loop belongs in the editor	38
8.2 The tightest CLI loop: ontology	39
8.3 The full registry: checks	40
8.4 Your own rules, without forking anything	40
8.5 The loop, drawn	42
8.6 Maturity checkpoint	42
9. Continuous integration	43
9.1 The mechanism	43
9.2 The problem: 296 findings	43
9.3 The obvious fix is wrong	43
9.4 The right fix: --own-namespace	44
9.5 Wiring it up	45
9.6 The one-command version	47
9.7 Across the boundary	47
9.8 Maturity checkpoint	47
10. Ingest and transform	48
10.1 Before the pipeline: drafting from a CSV	48
10.2 Stage 1: sketch — before any CSV is read	49
10.3 Stage 2: triplify — produce real RDF	49
10.4 Stage 3: data — assess what was actually produced	49
10.5 Whose problem is it?	51
10.6 What is not here	51
10.7 Maturity checkpoint	51
11. Release and change	52
11.1 Evidence-based semver: version-diff	52
11.2 Change impact: consistency	52
11.3 The migration annotation, and its direction	53
11.4 Applying repairs	53

11.5 The taxonomy layer: pattern-consistency	55
11.6 Governance: what the tooling does and does not do	56
11.7 Maturity checkpoint	57
12. Operate and consume	58
12.1 Runtime validation against a live store	58
12.2 Documentation as a product: docgen	59
12.3 Observability: not covered	60
12.4 Knowledge products and APIs: not covered	60
12.5 Maturity checkpoint	61
13. Coverage and gaps	62
13.1 Coverage by pipeline stage	62
13.2 Coverage by operating-model layer	62
13.3 Coverage of the ten SemOps elements	63
13.4 Rough edges, all encountered while writing this manual	63
13.5 The cheapest thing you are not doing	64
13.6 Gaps that have closed	65
13.7 What to do about the real gaps	65
14. Adoption roadmap	66
Phase 0 — Baseline before you build	67
Phase 1 — Reach Level 2: ontologies like code	67
Phase 2 — Reach Level 3: the gate	67
Phase 3 — Hold it: release discipline	68
Phase 4 — Level 4, and knowing when not to	69
The first ninety days, on one page	69
The shortest version	70
Appendix A — Diagram and brand conventions	71
A.1 The width rule	71
A.2 The label rule	71
A.3 The size rule	71
A.4 Colour: explicit fills, always	71
A.5 Checking a diagram before committing	72
A.6 Accessibility	72
A.7 When not to draw a diagram	73
A.8 The mark	73
A.9 The print edition	73
Index	74

Introduction

What this is

SemOps is the operational discipline required to build, deploy, govern and evolve semantic knowledge systems continuously — what DevOps did for software, applied to ontologies, mappings, rules and knowledge graphs.

There is a lot of SemOps *strategy* about: maturity models, operating models, nine-stage pipeline blueprints, polyglot technology menus. Strategy is cheap. The reason semantic programmes stall is almost never that nobody drew the diagram — it is that the diagram had no commands under it, no named owner, and no answer for the person who has to justify the spend in October.

This manual is the missing middle. It takes the SemOps strategy material as given and connects it, stage by stage, to **two toolchains that are built, tested and runnable today**:

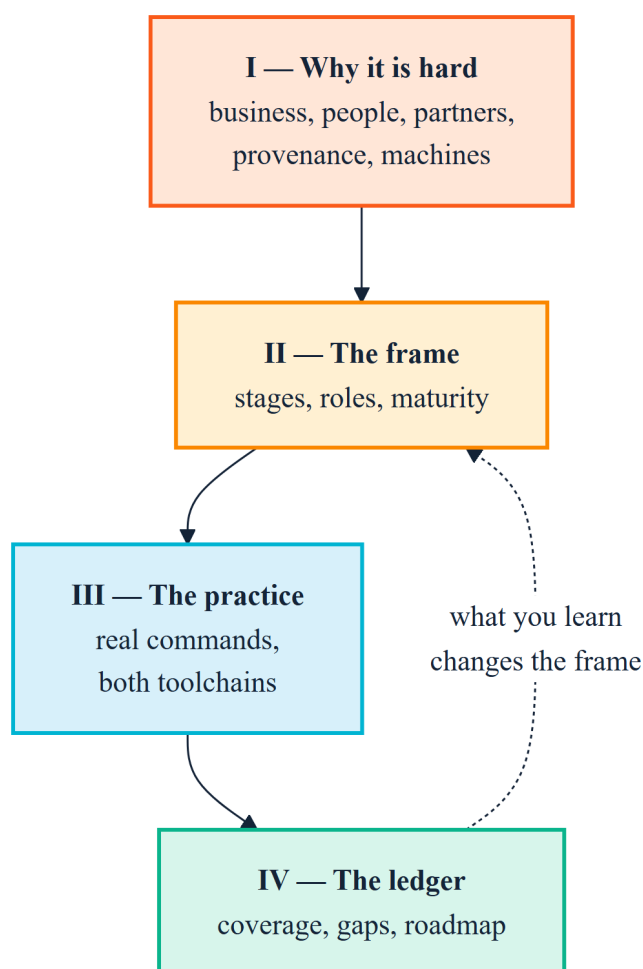
Toolchain	What it is	Where it fits
consolidated_ontology_suite_python	A Python CLI and library — 50-check registry, OWL2 reasoning,	Automation: CI gates, release control, pipelines

Toolchain	What it is	Where it fits
	version diffing, CSV→RDF triplification, auto-repair, live-triplestore checks. Published to PyPI as ontology-quality-suite	
consolidated_ontology_suite_webapp	The Ontology Development Suite VS Code extension — in-process SPARQL/SHACL/OWL2-RL engines via WASM, live diagnostics, graph view, query workbench	The author's desk: the loop before CI

Every command shown in Part III was **executed for real** while writing this manual, against the ontology suite's own bundled `examples/acme_robotics/` fixture. Where the output in this manual differs from what a strategy document would lead you to expect, the manual reports what actually happened and explains why — see Chapter 9 for the sharpest example, where the obvious flag is the wrong one.

Where the toolchain has no answer for a SemOps concept, this manual says so plainly and marks it grey. Chapter 13 is the honest ledger.

The shape of the argument



Part I is unusual in a technical manual, and it is first on purpose. The failure modes that kill semantic programmes are overwhelmingly organisational, cognitive and commercial rather than technical. A team that can run every command in Part III and has not read Part I will build something correct that nobody adopts.

Read it on paper

SemOps-Manual-A4.pdf — the whole manual as a 76-page A4 document: title page, contents with page numbers, every diagram rendered, and an index of commands, flags and check identifiers.

Rebuild it with `cd tools && npm install && node build-pdf.mjs` — see `tools/`.

How to read it

If you are deciding whether to fund this — Chapters 1, 3 and 4, then the coverage matrix in 13. About forty minutes.

If you have been handed a stalled semantic programme — Chapter 2 first. The diagnosis is usually there rather than in the technology.

If you are the engineer — skim 6 and 7, then work through Part III with a terminal open. Every command runs against fixtures bundled in the ontology suite, so nothing needs inventing to follow along.

If you are writing the business case — Chapters 1 and 14, and steal the maturity-level framing wholesale.

Conventions

Findings, commands and outputs in Part III are real. Where a number appears (*"296 findings"*, *"confidence 100%"*) it is the number the tool printed on the run described, not an illustrative figure.

Grey boxes and dashed borders in diagrams mean **not covered by this toolchain**. They are a deliberate, load-bearing part of the notation — see Appendix A.

Diagrams are Mermaid and render natively on GitHub. They are built to stay readable when printed A4 portrait; the rules that achieve that are documented in Appendix A for anyone extending the manual.

1. The business case

Part I — Why this is hard

The SemOps strategy material opens with capability: knowledge lifecycle management, CI/CD, observability, governance. All correct, and all of it answers the question "*what would good look like?*" — which is not the question anyone gets asked when they request budget. The question they get asked is "*what is broken today, what does it cost, and how will I know this fixed it?*"

This chapter is about answering **that** question, because semantic programmes fail at the funding stage far more often than at the modelling stage.

1.1 The four business problems that are actually semantic

Most data problems are not semantic problems and should not be sold as such. The ones that genuinely are share a signature: **the data is present, correct, and still produces the wrong answer, because two systems disagree about what a word means.**

Four patterns cover most of the real cases.

Reconciliation tax

Two departments report a number. The numbers differ. Nobody can say which is wrong, because "active customer" means *billed in the last 90 days* in finance and *logged in within 90 days* in product. Neither is wrong; they are answers to different questions wearing the same label.

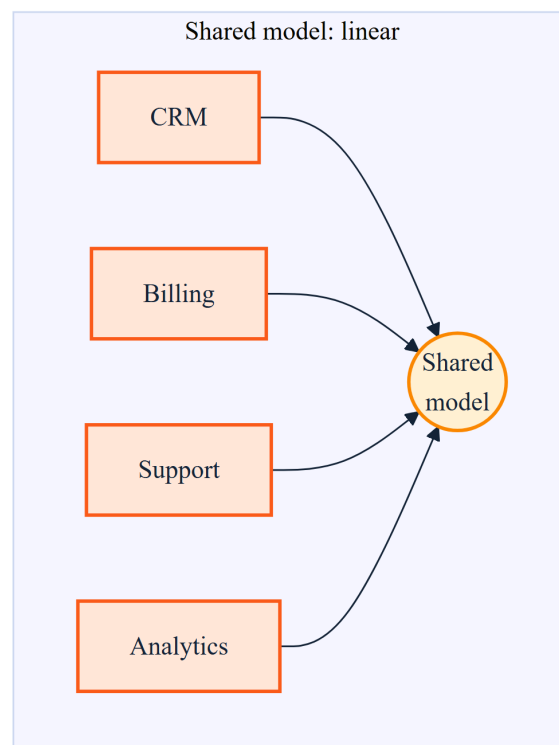
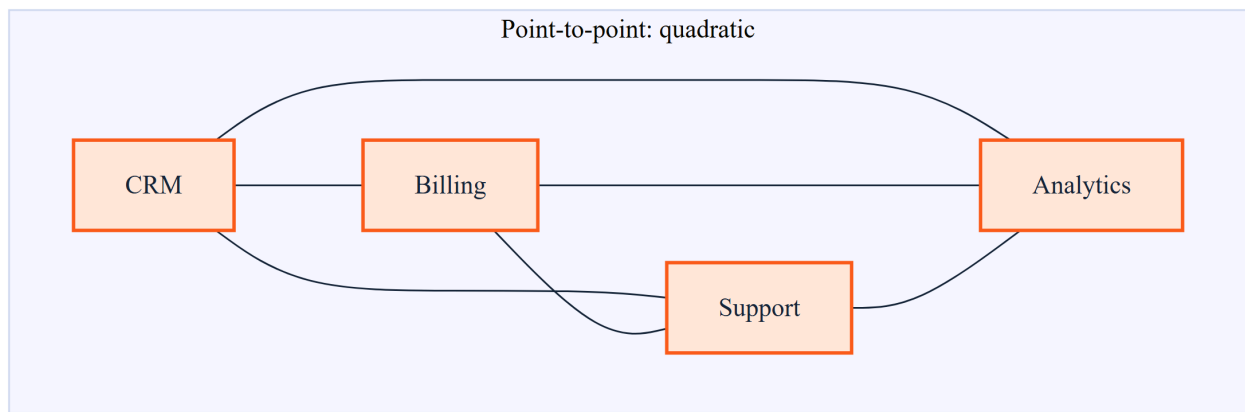
The cost is not the discrepancy. It is the **standing army of reconciliation** — the recurring analyst hours spent explaining the difference, the meeting that happens every month, the fortnight before every board pack. This cost is measurable, and almost nobody measures it, because it is distributed across dozens of people's calendars in slices too small to appear in any budget line.

Instrument it first. Before proposing an ontology, count the hours. Ask four analysts to tag a fortnight of calendar time as "reconciling definitions." The resulting number is usually large enough to fund the programme on its own, and — critically — it is a number that already exists in the business's own terms, rather than one you have to teach them to care about.

Integration cost that scales quadratically

Every new system integrated point-to-point with n existing systems costs n mappings. Ten systems is forty-five potential pairings; twenty is a hundred and ninety. Teams feel this as "integrations take longer every year" and diagnose it as a staffing problem.

A shared model changes the exponent, not the constant. Each system maps once, to the model. The pitch is not "we will make this integration cheaper" — it will not, the first one is *more* expensive — it is **"we will stop the curve bending."**



The honest caveat, which you should say out loud before someone else does: the shared model is itself an asset that must be maintained, and an unmaintained shared model is worse than point-to-point mappings because it carries authority it no longer deserves. That maintenance burden is precisely what the rest of this manual automates — it is the reason SemOps exists as a discipline rather than as a modelling exercise.

Provenance you cannot reconstruct

A regulator, an auditor, or a customer asks "*where did this figure come from?*" and the answer takes three weeks and two people who happen to remember. In regulated sectors this is not an efficiency problem, it is an existential one: the inability to evidence lineage can invalidate the figure regardless of whether it was correct.

This is the pattern with the clearest financial framing, because the downside is a number someone has already written down — the fine, the qualification, the remediation programme.

Knowledge that leaves with the person

The system works because Margaret knows that the `STATUS` field means something different for records created before the 2019 migration. Margaret retires in March. Nothing in any system records what Margaret knows.

The failure mode is delayed and therefore rarely attributed to its cause: the programme that breaks in eighteen months' time is not connected, in anyone's mind, to the retirement that caused it.

1.2 Why the payoff is invisible

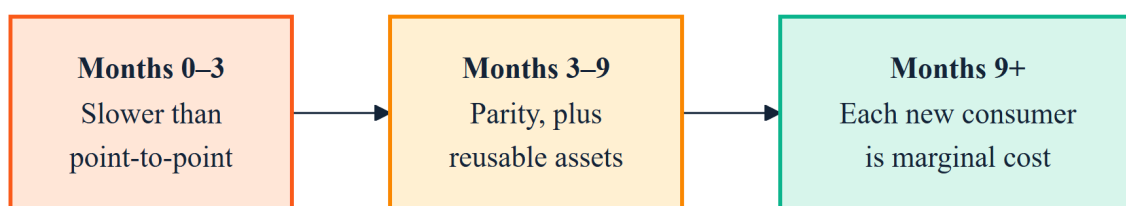
Semantic work has an awkward economic shape. Done well, its payoff is **a cost that does not arrive** — the outage that did not happen, the reconciliation meeting that got cancelled, the integration that took two weeks rather than two months.

Organisations are structurally poor at valuing avoided costs. The team that prevents ten incidents is less visible than the team that heroically resolves one. This is not irrationality; it is a measurement problem, and it has three practical consequences worth planning around.

Baseline before you build. Once the ontology is in place you can no longer measure what it saved, because the counterfactual is gone. The reconciliation hours, the mean integration lead time, the mean time to answer a lineage question — capture them *before* anything changes, or you will spend the rest of the programme asserting benefits you cannot evidence.

Pick a first use case with a visible failure. Not the most valuable one; the most *legible* one. A use case whose current failure mode is embarrassing and specific — a regulator's question that took three weeks — produces a before/after story an executive can retell. A use case whose failure mode is diffuse inefficiency produces a slide nobody remembers.

Expect the trough. The first delivery is slower than the point-to-point alternative, because you are building the shared asset and the first consumer of it simultaneously. Say so at the start. A programme that promised acceleration and delivered a trough loses its sponsor in month four; one that predicted the trough and hit it on schedule gains credibility.



1.3 The decay problem, which is the SemOps problem

There is one failure mode that dwarfs the others, and it is the reason this manual exists.

An ontology is most accurate the day it ships. Every day after that, without deliberate maintenance, it decays.

Enterprise ontologies fail in a small number of well-documented ways, and almost all of them are variations on decay:

Failure mode	What it looks like eighteen months in
Business-model drift	Definitions that were precise last year are now misleading — the product line changed, the model did not
Unknown unknowns	Expert interviews captured what experts knew; what nobody thought to mention is now baked into the structure
Maintenance fatigue	Domain experts stop accepting refinement-workshop invitations. Calendar declines end the programme, not a decision
Cross-domain collision	Two separately-built ontologies must reconcile; the integration costs more than either original

Failure mode	What it looks like eighteen months in
Governance abandonment	The project ends, the owner moves on, no feedback loop remains, drift is now silent

Read that table as a specification. Each row is a requirement:

- Drift must be **detected mechanically**, because humans will not notice it → Chapter 11, version-diff and consistency.
- Maintenance must cost **minutes, not workshops**, or fatigue wins → Chapter 8, the in-editor loop.
- Breakage must be **caught at the gate**, not in production → Chapter 9, --fail-on Violation.
- Ownership must be **named and enforced by the pipeline**, not by goodwill → Chapter 6, roles.

This is the actual argument for SemOps, and it is worth stating baldly to a sponsor: **the risk is not that you build the wrong ontology. It is that you build the right one and then let it rot**, at which point it is worse than nothing, because systems and people now trust it.

1.4 What to write on the slide

Three sentences, in this order, is usually enough:

1. *"Today, [specific, instrumented cost] — N analyst-hours a month, or a three-week lineage question, or an integration curve that has bent 40% in two years."*
2. *"We will fix it for one legible use case, and we expect to be slower for the first quarter."*
3. *"The ongoing cost is automated maintenance, not a standing modelling team — here is the pipeline that does it."*

Sentence 3 is the one this manual equips you to make truthfully. Without it, sentence 1 has been pitched — and declined — in most large organisations already.

Sources

- [How Enterprise Ontologies Fail, And How to Stop It — Modern Data 101](#)
- [Enterprise Knowledge Graph Use Cases & Adoption Strategies 2026](#)
- [Semantic Web and Knowledge Graphs for Industry 4.0 — MDPI](#)

2. People and cognition

Part I — Why this is hard

Semantic technology has an unusual property among enterprise technologies: **its artefacts are arguments about meaning, and meaning is political**. A schema change is a technical act. A definition change is a negotiation about whose account of the business is correct.

This chapter is about the human system the toolchain has to survive contact with. It is not soft-skills garnish appended to the engineering — it determines which technical choices are viable. The reason Chapter 9 recommends a particular CI flag over the obvious one is, in the end, a cognitive argument as much as a correctness one.

2.1 Definitions are territory

When finance and product disagree about "active customer," the disagreement is not a misunderstanding to be cleared up. Each definition is load-bearing for someone's targets, someone's forecast, someone's bonus. Asking the two teams to "agree a single definition" asks one of them to accept a number that makes their performance look worse.

The naive semantic response — *both can exist, we will model them as distinct subclasses with clear labels* — is technically correct and socially incomplete. It resolves the representation problem while leaving the actual question untouched: **which one goes in the board pack?**

Three practical consequences:

Model the disagreement, do not resolve it. BilledActiveCustomer and EngagedActiveCustomer as distinct, well-labelled terms is a better outcome than a compromise definition that neither team uses. The ontology's job is to make the distinction visible and unambiguous, not to adjudicate it.

Get the adjudication done by someone with the authority to do it. If the programme requires a decision about which definition is canonical, that decision needs an owner senior enough to make it stick. A knowledge engineer does not have that authority and should not be manoeuvred into pretending they do — the common failure is a modeller quietly picking one, shipping it, and discovering six months later that half the business has been ignoring the model.

Record who decided, and when. Definitions get relitigated. Without the decision trail, the same argument recurs annually with fresh participants and no memory of why the current answer was chosen.

2.2 The priesthood problem

Semantic technology attracts a particular failure: a small group becomes fluent in OWL, SHACL and SPARQL, and gradually becomes the only group that can read the organisation's most important asset.

This feels like success — the team is indispensable — and it is the surest predictor of decay. It produces:

- **A review bottleneck.** Every change queues behind the same three people.
- **Unchallengeable models.** Domain experts cannot read the artefact well enough to notice it is wrong, so they stop trying, and the modelling loses its only real error-correction mechanism.
- **A bus factor of two,** applied to the asset the organisation was told to treat as strategic.
- **A veneer of consultation.** Workshops happen; experts nod at diagrams they cannot verify; the model records the modeller's understanding, not the expert's.

The counter-measure is not "train everyone in OWL." It is to ensure **the artefacts that domain experts must validate are rendered in a form they can actually validate**, and that the formal artefact is generated from or checked against that form. Concretely, in this toolchain:

- Generated reference documentation, not raw Turtle, is what goes to a domain expert for review (Chapter 12, docgen).
- The extension's hierarchical outline and graph view render structure visually — a domain expert can see that Contractor sits under Employee and say "no, it does not," without reading a single axiom (Chapter 8).

- Check findings carry a human-readable message and remediation field, not just a rule identifier, so a non-specialist can tell whether a finding matters.

A useful test: pick a domain expert who has never written RDF and ask them to find the deliberate error in your model using only the artefacts you would normally send them. If they cannot, your review process is theatre, and the priesthood has already formed.

2.3 Cognitive styles: people do not categorise the same way

Ontology work assumes a particular cognitive style — comfort with abstraction, hierarchy, and the discipline of saying only what is formally true — and then treats people who do not share it as obstacles. They are not; they are the domain.

Four styles show up reliably in modelling sessions. None is wrong, and a session that only accommodates the first will produce a model that is formally elegant and factually incomplete.

Style	How they answer " <i>what is a Customer?</i> "	What they contribute	How they get lost
Taxonomic	Starts with the general case, refines downward, wants the hierarchy settled first	Structural coherence; spots inconsistent partitioning	Over-abstracts; builds levels nobody needs
Narrative / procedural	" <i>Well, when an order comes in...</i> " — tells you the process, not the type	Real edge cases; how the business actually behaves	Frustrated by requests to state things atemporally
Exemplar-driven	" <i>Acme Ltd is a customer. So is Bob, sort of.</i> " — reasons from instances	Fast falsification of an over-clean model	Resists generalising; every case is special
Formal / logical	Asks whether the classes are disjoint and whether the property is functional	Precision; finds the contradictions	Optimises for logical tidiness over usefulness

The practical move is to **elicit in the style the expert has, and translate**. An exemplar-driven expert should be shown instances, not a class hierarchy. A narrative expert should be allowed to tell the story, with the modeller extracting types afterwards and playing them back as a story to confirm.

This is also why the fixture in Part III is a small org chart rather than an abstract domain. `Employee`, `Contractor`, `worksIn` are things every style can reason about, which makes it a fair test of tooling rather than a test of whether the reader thinks like a taxonomist.

A specific trap worth naming: **the exemplar-driven expert is usually right and usually ignored**. When someone says "Bob is sort of a customer," they have found a genuine boundary problem. The taxonomic instinct is to rule it out of scope. Eighteen months later, "sort of" cases are 15% of the data and the model has no place to put them.

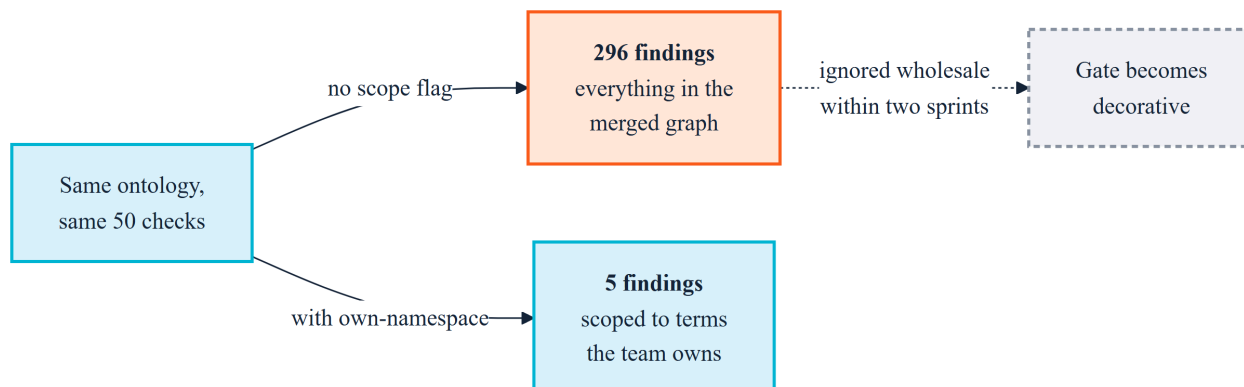
2.4 Cognitive capacity is a hard budget

Working memory holds roughly four items. Not four ontology classes — four *chunks*, where a chunk is whatever the person has already internalised well enough to treat as one thing. An experienced modeller may hold "the employment subtree" as one chunk; a newcomer holds five classes and has nothing left over to reason with.

Every design decision in a semantic toolchain either spends or conserves this budget. Three places where the spending is visible and avoidable:

The 296-finding report

Running the full check registry over the manual's fixture with its real org: and FOAF imports resolved produces — verified, not illustrative — **296 findings**. Adding a single flag that scopes the report to the terms the team actually owns produces **5**.



Both numbers are correct. The 296-finding run is genuinely checking every triple in the merged graph, including the internal documentation habits of the W3C Organization Ontology. It is also **cognitively useless as a gate**: no team triages 296 findings per commit, so within two sprints the gate is either switched off or ignored, and the four findings that were actually theirs are lost with the rest.

This is the single most important practical lesson in the manual, and Chapter 9 works through it with the real outputs — including why the *obvious* way to cut the noise introduces four false findings of its own.

Ontology size versus the review budget

A 40-class ontology can be reviewed by a domain expert in a sitting. A 400-class ontology cannot be reviewed by anyone, in any sitting, ever — it can only be spot-checked. Past that scale, correctness stops being a function of human attention and becomes entirely a function of mechanical checking. Teams routinely cross this threshold without noticing that their quality model has silently changed.

Error messages as cognitive artefacts

LOG-001 means nothing. *"Class acme:Contractor is asserted disjoint with its own transitive superclass acme:Employee, which makes acme:Contractor logically unsatisfiable"* — followed by *"remove either the subclass axiom or the disjointness axiom; the two together are contradictory"* — means something to a competent engineer who has never studied description logic. The second form is what the suite actually emits, and it is the difference between a check that teaches and a check that gates.

2.5 The workshop is a social event with a technical output

Domain experts have finite patience for refinement workshops, and maintenance fatigue is a documented killer of enterprise ontology programmes. Calendar declines end more semantic initiatives than technical failures do.

Design accordingly:

- **Front-load the expensive human time; automate the recurring part.** The first elicitation genuinely needs a room. The 400 subsequent consistency questions do not, and if you spend expert goodwill on them, it will not be there when you need it for the questions only a human can answer.
- **Bring falsifiable artefacts, not blank paper.** A wrong draft gets corrected in ten minutes; an open question gets a forty-minute discussion and no decision. Experts are far better critics than authors — this is not a failing, it is how expertise works.
- **Close the loop visibly.** An expert who never sees their correction land concludes, correctly, that the session was extractive. Show the diff.

2.6 What this implies for the toolchain

Everything above resolves into four requirements that Part III should be read against:

Human problem	Toolchain requirement	Where it lands
Priesthood, unreadable artefacts	Generated docs and visual structure for non-specialists	Ch. 8, Ch. 12
Cognitive overload at the gate	Findings scoped to what the team owns	Ch. 9
Maintenance fatigue	Drift detected mechanically; repairs proposed automatically	Ch. 11
Territorial definitions	Change impact made explicit before merge, so decisions have evidence	Ch. 11

If a tool does not reduce the load on the humans, it will be abandoned regardless of correctness. That is the whole argument, and it is worth carrying into every tooling decision the programme makes.

Sources

- [How Enterprise Ontologies Fail, And How to Stop It — Modern Data 101](#)
- [Enterprise Knowledge Graph Platform Comparison 2026 — OvalEdge](#)
- [My Ontologist: Evaluating BFO-Based AI for Definition Support \(arXiv\)](#)

3. Across the boundary: peers and supply chains

Part I — Why this is hard

Everything in Chapter 2 assumed a shared employer: one budget, one CI system, and — ultimately — someone senior enough to settle an argument. Cross out those three assumptions and you have the cross-enterprise case, which is where semantic technology is simultaneously most valuable and most likely to fail.

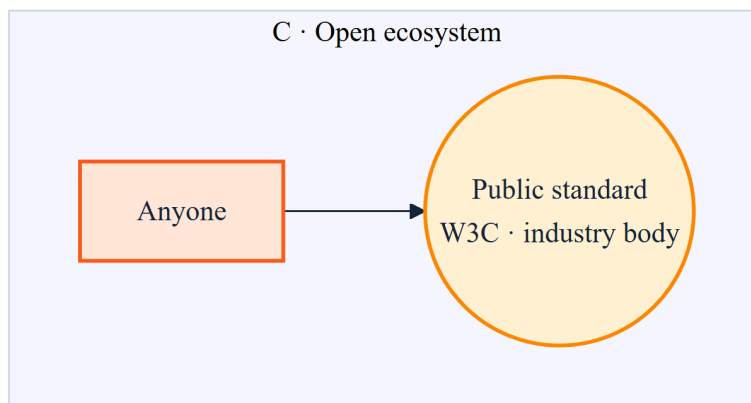
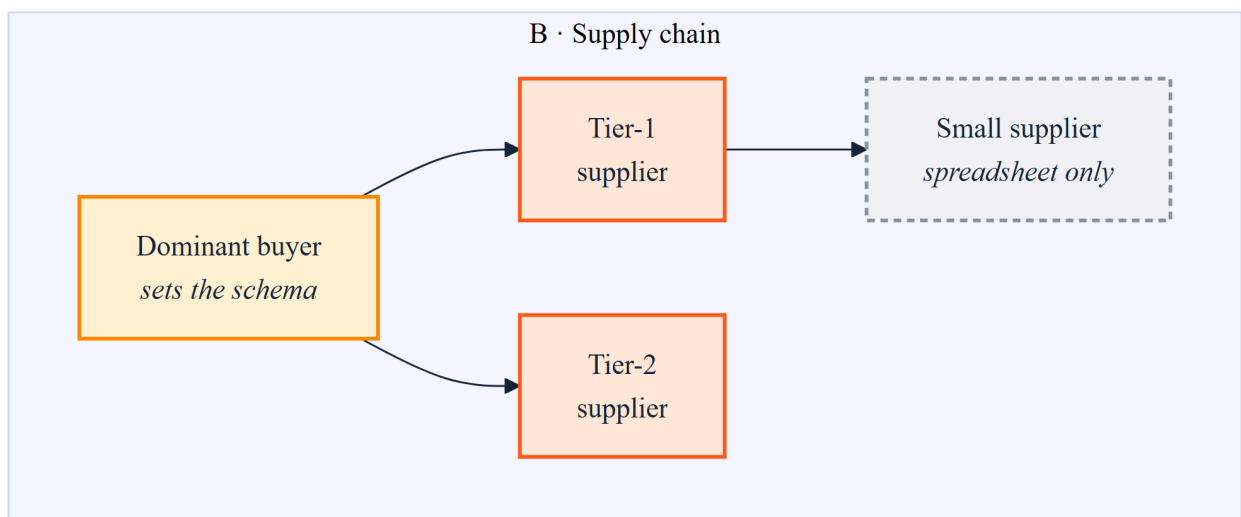
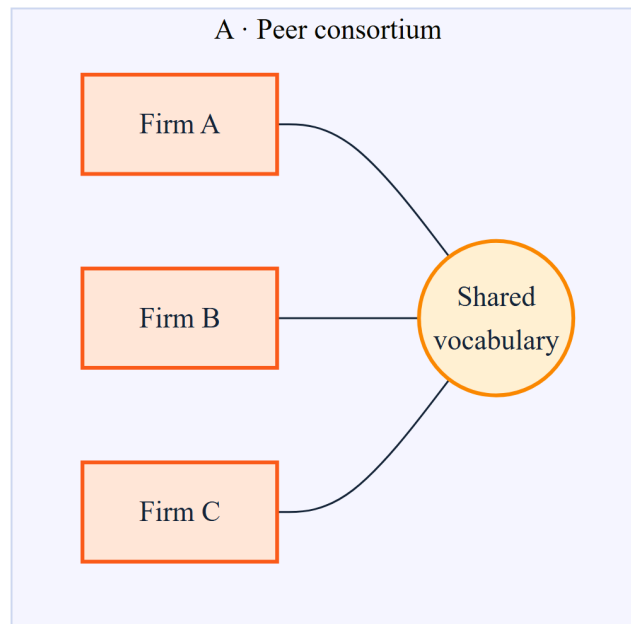
It is most valuable because shared meaning across organisations cannot be achieved by a shared database — nobody will accept one. It is most likely to fail because **every enforcement mechanism SemOps relies on stops at the company boundary**. You cannot fail someone else's build.

3.1 What changes at the boundary

Inside one enterprise	Across enterprises
One CI system can gate every change	You cannot gate a partner's pipeline; you can only detect and notify
A senior owner can settle a definition dispute	No shared authority — disputes are settled by negotiation or by power
Breaking changes can be fixed downstream by the same team	Downstream is someone else's backlog, with its own priorities
Cost of coordination is internal overhead	Cost of coordination is a commercial term someone must agree to pay
Sharing a model is uncontroversial	A model can reveal commercially sensitive structure
Capability is roughly uniform	Partners range from a semantics team to a spreadsheet and one IT contractor

The last row is the one most often missed, and it dominates supply-chain work.

3.2 Three topologies, three different problems



A. Peer consortium — the coordination problem

Competitors or near-peers agree a shared vocabulary: trade associations, regulatory reporting groups, interbank messaging, research consortia.

The hard parts are not technical:

- **Nobody can compel anyone.** Progress is by consensus, which means the model drifts towards the union of everyone's requirements and the intersection of everyone's willingness to constrain. Consortium ontologies are characteristically under-constrained — everything is optional, because making anything mandatory requires unanimity.
- **Commercial sensitivity has a modelling shadow.** How you model your customer segmentation reveals your customer segmentation. Members will decline to model the very areas where alignment would be most valuable, and will not always say that is why.
- **Competition-law caution is real** and worth taking seriously rather than routing around. Legal review of a shared model in a concentrated market is a legitimate step, not obstruction.
- **Release cadence is set by the slowest member**, because a change nobody has implemented is not a change.

What works: a deliberately small mandatory core, with extension points. Publish the core with real constraints; let each member extend in their own namespace. This is exactly the pattern Part III uses at company scale — import the shared vocabulary, own your extensions, and check only what you own (Chapter 9).

B. Supply chain — the capability-asymmetry problem

One organisation has the leverage to mandate a schema — a large manufacturer, a retailer, a public body. Compliance is a condition of doing business, so adoption is not the problem.

The problems are:

- **Capability asymmetry.** A tier-1 supplier may have a data team. Their sub-supplier has a spreadsheet and a part-time IT contractor. A mandate that assumes RDF fluency does not produce compliance; it produces a cottage industry of consultants filling in forms on suppliers' behalf, and the data quality that implies.
- **The mandate arrives without the tooling.** "Send us this in the standard format" is not an ask, it is a project — and the cost lands on the party least able to absorb it.
- **Verification is one-directional.** The buyer validates what arrives. The supplier gets a rejection with no way to test in advance, so the loop is: send, get rejected, guess, resend.

What works: **give the supplier the gate.** The single highest-leverage intervention in supply-chain semantics is publishing the validation as something the supplier can run themselves, before submitting. Concretely, from Part III's toolchain:

- Publish your SHACL shapes and check registry as files, not as prose in a PDF. The suite reads a registry directory from `--registry/--shapes/--sparql`, so a shared registry is a distributable artefact (Chapter 8).
- Accept CSV. Most of the supply chain has CSV, and the CSV→RDF path (Chapter 10) means the supplier never has to author RDF at all — they produce the tabular extract they already have, and the transformation is yours to own and version.
- Ship the check as an editor experience, not a batch report. The VS Code extension runs its checks in-process via WASM, with no Python or Java runtime required — which for a small supplier is the difference between "we can run this" and "we need to procure something."

That last point deserves emphasis: **the runtime dependency is a governance decision.** Mandating a validation step that requires a JVM excludes a class of suppliers from self-service compliance as surely as mandating a language they do not speak.

C. Open ecosystem — the stability problem

You depend on a public vocabulary you do not control: FOAF, the W3C Organization Ontology, schema.org, an industry standard.

Here the risk inverts. You are not coordinating; you are **exposed**. The vocabulary may change, stagnate, or turn out to encode conventions that break your tooling in ways nobody documented.

Part III contains a verified, live example of exactly this. FOAF deliberately declares `foaf:name` with `rdfs:domain owl:Thing` — an intentional "usable on anything" convention. A conformance checker that walks asserted `rdfs:subClassOf` edges does not know that every class is implicitly a subtype of `owl:Thing` under OWL semantics, because that is an axiomatic fact rather than an ordinary triple. The result is a domain/range "violation" on every single `foaf:name` triple in otherwise perfect data (Chapter 10).

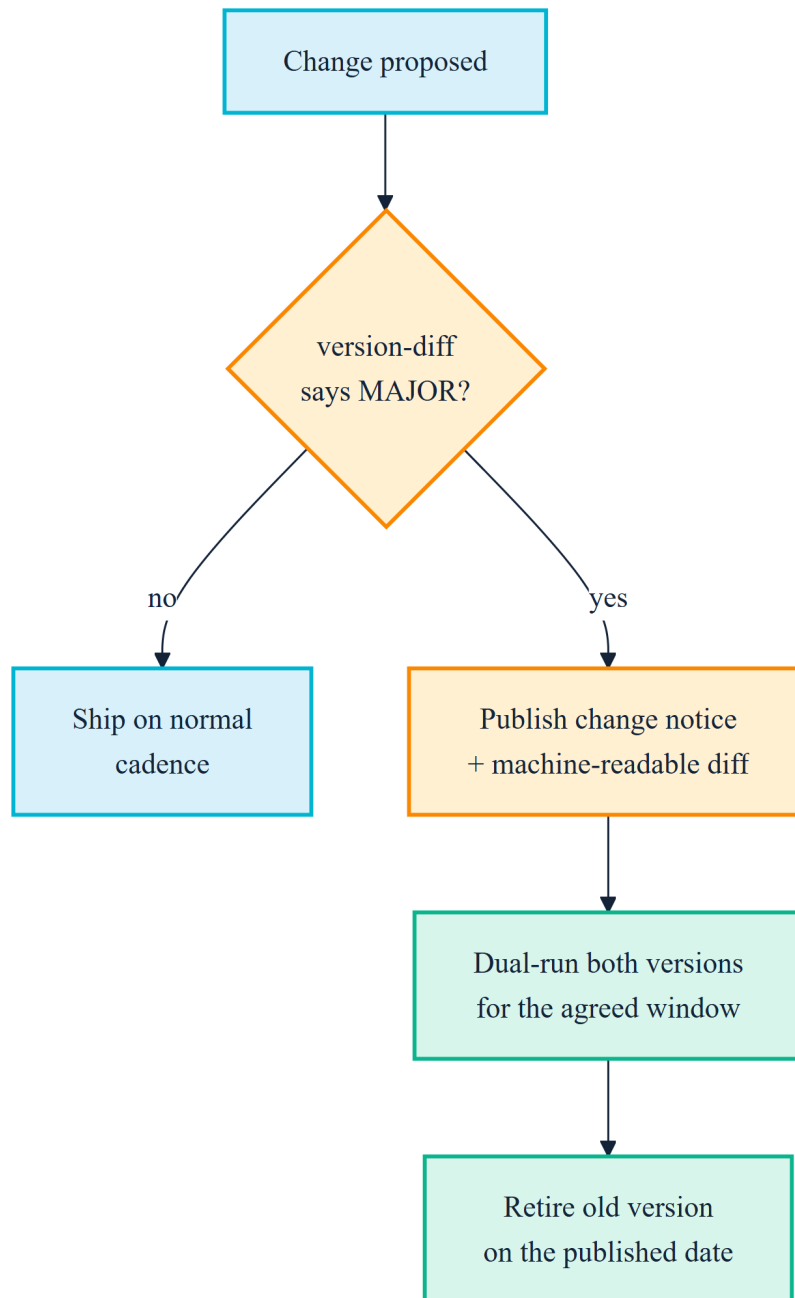
Nobody is at fault. FOAF is behaving as designed, the checker is behaving as designed, and the finding is still wrong. **Depending on a public vocabulary means inheriting its conventions, including the ones that were never written down.** Budget for discovering them the hard way.

3.3 The cross-boundary release problem

Inside one enterprise, a breaking ontology change is manageable: detect it, find the affected queries, fix them, ship together (Chapter 11 does all three, and the last one automatically).

Across a boundary, the same change is a **negotiation with a lead time**. You cannot auto-repair a partner's queries; you often do not know they exist.

The protocol that works looks like this:



Two things make this workable rather than aspirational:

The bump decision must be evidence-based, not a judgement call. `version-diff` classifies a change by inspecting what actually happened to the axioms — a removed class is MAJOR whatever the author intended. In Chapter 11 it correctly returns MAJOR for the fixture's release, on the evidence of one removed class, while still reporting the additive changes as minor. Across a boundary this matters enormously, because "is this breaking?" stops being a conversation and becomes a command whose output both parties can run.

The change notice should carry the machine-readable diff, not just prose. `version-diff --json` writes `diff.json` alongside the human-readable text. A partner can then diff programmatically rather than reading your release notes and hoping.

Migration annotations are a cross-boundary contract

When you rename a term, assert the migration annotation **from the retiring IRI to the new one**:

```
acme:Engineer owl:equivalentClass acme:SoftwareEngineer .
```

owl:equivalentClass is logically symmetric, so a reasoner treats both directions identically — but rename *detection* reads it directionally, from the removed term to its replacement. Written the other way round, the tooling falls back to name-similarity guessing and confidence drops sharply.

Inside one company that costs you an afternoon. Across a boundary it is the difference between a partner's tooling automatically identifying the replacement, and a human reading a PDF. Write the annotation in the direction the tools read.

3.4 Trust, provenance and attribution

Once data crosses an organisational boundary, "*who said this?*" becomes as important as "*what does it say?*" — and it is a question RDF is unusually well equipped to answer, provided someone decided to answer it.

- **Keep each partner's contribution in its own named graph.** This is the single decision that most improves cross-enterprise operability, and it is nearly free at ingest time and nearly impossible to retrofit. The suite's live-triplestore checking is named-graph aware precisely because this is the expected topology (Chapter 12).
- **Do not merge on ingest.** A merged graph cannot answer "which supplier asserted this?", and that is the question the incident review will ask.
- **Record the conformance verdict, not just the data.** "This batch passed the agreed shapes at version 2.1 on this date" is the artefact that resolves disputes.

3.5 Where this leaves the practice

The cross-boundary case does not need a different toolchain. It needs the same toolchain with three shifts of emphasis:

Inside	Across
The gate fails the build	The gate produces a notice the other party can act on
Repairs are applied automatically	Repairs are published as patches for the other party to apply
Ownership is a role	Ownership is a contract term
Checks scoped to what we own	Checks scoped to what we own and published so partners can run them too

The right-hand column is mostly a matter of *who receives the output*, which is why the mechanics in Part III transfer largely unchanged. The failure mode is assuming the left-hand column's enforcement model still applies — building a beautiful gate, and discovering it has no jurisdiction.

Sources

- [Semantic Web and Knowledge Graphs for Industry 4.0 — MDPI](#)
- [An ontology-based method for knowledge reuse in the design for maintenance of complex products — ScienceDirect](#)
- [How Enterprise Ontologies Fail, And How to Stop It — Modern Data 101](#)

4. From research to industry

Part I — Why this is hard

Semantic technology has deeper academic roots than almost any other part of the enterprise data stack. Description logics, OWL, SHACL, the reasoners, and a large share of the libraries all originate in university research groups. This is a genuine strength — the formal foundations are unusually solid — and it creates a characteristic and under-discussed problem:

A research project and an industrial product optimise for different things, and the artefacts they produce are correspondingly different. Adopting a research artefact as though it were a product is one of the most reliable ways to derail a semantic programme.

This chapter is about that mismatch, and about how to consume research output responsibly rather than either naively or dismissively.

4.1 Different objective functions

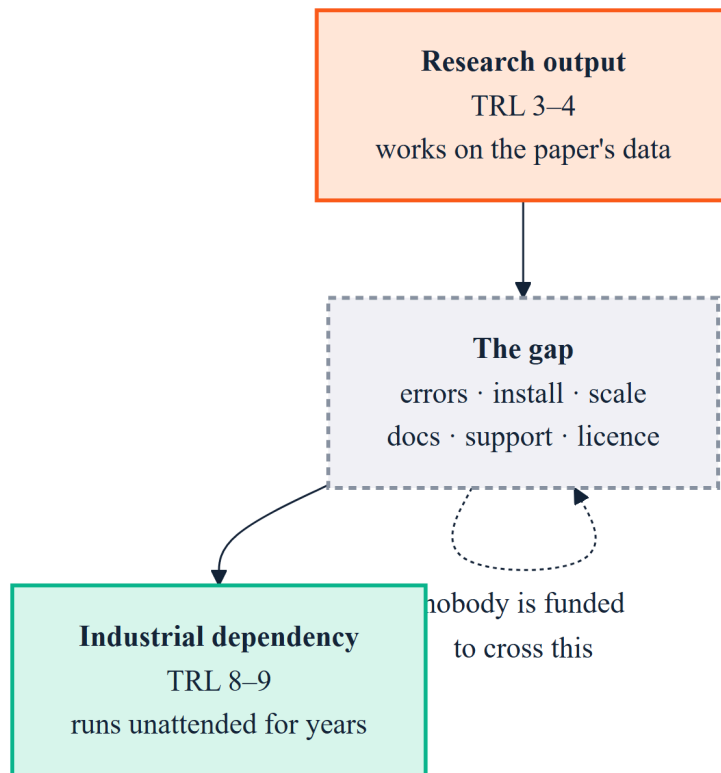
Neither party is doing anything wrong. They are being rewarded for different outcomes.

	University project	Industrial product
Success is	A novel, defensible contribution, published	A system that runs unattended for years
Rewarded for	Novelty, generality, formal results	Reliability, predictability, supportability
Timeline	Funded 3 years; ends on a date	Indefinite; cost accrues after launch
Team continuity	The PhD student graduates	Deliberate handover and on-call
Failure handling	Out of scope; the paper reports the good case	The main event; most code is edge cases
Evaluation	Precision/recall against a gold standard	Did the build stay green, did anyone have to think
Docs aimed at	Reviewers who know the field	A newcomer at 4pm on a Friday
"Done"	Paper accepted	Never

The consequential row is **timeline**. A research artefact's maintenance model is usually *"the author will fix it if they have time and still care."* That is not a criticism — no grant funds a decade of maintenance — but it means the artefact carries an implicit expiry date that does not appear anywhere in its README.

4.2 What actually arrives

A research artefact typically arrives at roughly **TRL 3–4**: validated in a lab setting, demonstrated on the paper's own dataset. Industrial dependency requires roughly **TRL 8–9**. That is not a small increment; it is usually more work than the original research.



Concretely, the things usually missing:

Failure behaviour. Research code tends to assume well-formed input, because the paper's dataset was well-formed. Production input is not. The difference shows up as stack traces rather than diagnostics.

Installation as a solved problem. "Requires Java 8, this specific JAR, and these three environment variables" is acceptable in a lab and is a procurement event in an enterprise.

Scale characteristics. Evaluated on 10,000 triples; your graph has 400 million. Reasoning in particular has complexity characteristics that make this gap qualitative rather than quantitative.

Error messages for non-specialists. Compare LOG-001 with the suite's actual message — *"Class acme:Contractor is asserted disjoint with its own transitive superclass acme:Employee, which makes acme:Contractor logically unsatisfiable"*, plus a remediation line. The first is a research artefact's idiom; the second is a product's.

Licence and provenance clarity. Research code frequently has no licence file at all, which means it is not usable, which is rarely what the author intended.

Backwards compatibility as a commitment. Research artefacts change shape between versions because the research changed shape. That is correct behaviour for research and disqualifying for a dependency.

4.3 A verified example from this manual's own toolchain

This is not hypothetical, and it happened while writing this manual.

The ontology suite supports real OWL2 DL reasoning through external reasoners — HermiT and Pellet, both research-lineage tools — as an **optional extra**. Running the quality gate on the fixture without that extra installed produced, verbatim:

LOG-001	logical	Class disjoint with its own ancestor	Violation
REA-022	reasoning	External DL reasoner unavailable -- only owlrl-based checks ran	Info

with this message on REA-022:

The external DL reasoner ('hermit') could not be run: cannot access local variable 'entity' where it is not associated with a value. Only the always-on owl RDFS/OWL2-RL closure and pattern checks (REA-001..004, LOG-001..007) ran. These are sound but not complete for full OWL2 DL — a class can be genuinely unsatisfiable without any of them firing.

Three things are worth extracting from that.

The research-grade component failed in a research-grade way. An internal Python `UnboundLocalError` surfacing through the reasoner bridge is not a diagnosable error for the team using it. There is nothing to act on in it.

The industrial wrapper turned that into a usable signal. Rather than crashing, or — much worse — silently reporting "no problems found," the suite emitted a first-class finding that states exactly which checks did not run, what that means for the soundness/completeness of the result, and what to install to fix it. The distinction between "*we checked and it is fine*" and "*we could not check*" is preserved, and that distinction is the entire difference between a gate you can trust and one you cannot.

The dependency was optional by design. The DL reasoner lives behind `uv sync --extra reasoner` and requires a Java runtime. The core gate — the always-on owl closure plus pattern checks — has no such dependency, and it caught the deliberate contradiction in the fixture anyway, as LOG-001. The research-grade capability is available to those who want it and load-bearing for nobody.

That is the pattern to copy, and it generalises past this one tool:

Consume research artefacts as optional enhancements behind a stable interface, with a supported fallback that is sufficient on its own. Never let the research-grade component sit on the critical path.

There is a second layer to the same story. Even when the DL reasoner *is* installed and working, it has real limitations of its own — the suite's own reasoning documentation records a case where Hermit trips on an ontology using `xsd:date`. A tool being research-grade does not mean it is unreliable; it means its failure modes are yours to discover, contain and route around.

4.4 The evaluation mismatch

The two worlds do not even measure the same thing.

Research asks: *does this technique find the errors, on a gold-standard corpus, better than the prior technique?* Precision and recall against a labelled set.

Industry asks: *did this stop a bad change reaching production, without costing the team more attention than the bad change would have?*

The second question has a term the first does not: **the human cost of the output**. A checker with better recall that produces 296 findings where 5 are yours (Chapter 2) scores well on the research metric and fails in practice, because a gate nobody triages is not a gate.

This is why Chapter 9's central recommendation is about *scoping the report*, not about check quality. The checks were already good. The industrial problem was never detection; it was attention.

4.5 How to consume research output well

Neither naive adoption nor blanket refusal. Six practices:

1. **Pin exactly.** Research artefacts change shape between versions. Pin the version and upgrade deliberately.
2. **Wrap behind your own interface.** Never let a research library's API become your internal API, or you inherit its churn.

3. **Have a fallback that is sufficient alone.** As above: the optional extra is an enhancement, never the critical path.
 4. **Convert its errors into your diagnostics.** REA-022 above is the model — catch the incomprehensible failure, state what did not run and what that means.
 5. **Check the licence before the capability.** An unlicensed artefact is not adoptable however good it is. Establish this first; it is cheap and it is occasionally fatal.
 6. **Budget for the crossing.** If the artefact is load-bearing, someone must be funded to maintain the industrial wrapper. If nobody is, you have a dependency with an expiry date and no calendar entry.
-

4.6 The other direction

The traffic is not one-way, and industry's contribution is usually the unglamorous half:

- **Real corpora.** Research is starved of realistic, messy, licensable data.
- **Failure reports.** The `xsd:date` reasoner limitation, the FOAF `owl:Thing` domain convention in Chapter 10, the SHACL engine that ignores `sh:severity` inside SPARQL-based constraints — these are findable only by running tools against real work.
- **Scale evidence.** The engine benchmarks in the suite's architecture documentation exist because someone ran both engines on a real-sized ontology and wrote down what happened.

The healthy relationship is: research supplies the formal foundations and the novel techniques; industry supplies the corpora, the failure reports and the maintenance. Both halves are needed, and the second half is the one that determines whether any of it survives contact with an actual enterprise.

Sources

- [Closing the gap between software engineering education and industrial needs \(arXiv\)](#)
- [Generative AI solutions for software quality: Assessing industrial readiness — Software Quality Journal](#)
- [A core ontology for modeling life cycle sustainability assessment on the Semantic Web — Journal of Industrial Ecology](#)
- [Are Computer Science and Engineering Graduates Ready for the Software Industry? \(arXiv\)](#)

5. GenAI and agents

Part I — Why this is hard

Generative AI affects semantic practice in two directions at once, and they are usually discussed as if they were one thing:

- **AI for semantics** — large models help *build* ontologies: reading documents, proposing classes, drafting definitions and constraints.
- **Semantics for AI** — ontologies and knowledge graphs *ground* AI systems, giving agents typed, validated, authoritative structure to act against.

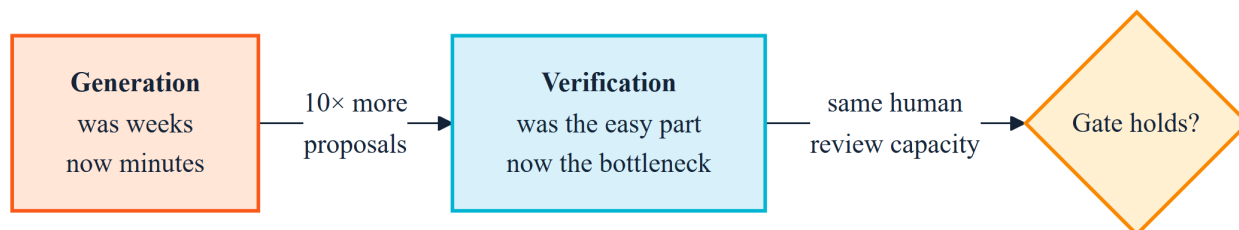
Both are real. Both change the economics of the work. Neither removes the need for the discipline in this manual — and the second direction, in particular, raises the value of it sharply.

5.1 AI for semantics: generation gets cheap

Large models are genuinely good at the tasks that used to consume the opening months of an ontology project: reading a corpus and extracting candidate terms, proposing class hierarchies, drafting definitions in consistent prose, generating competency questions, suggesting where a constraint might belong, translating between serialisations, and explaining an unfamiliar vocabulary to a newcomer.

This is not marginal. It removes a genuine barrier — the specialist bottleneck described in Chapter 2 — and lets domain specialists outside computer science participate in modelling their own domains.

But notice precisely what has changed. **The cost of producing a candidate model has collapsed. The cost of knowing whether it is right has not.**



A model's output is *plausible by construction* — that is what it optimises. In ontology work plausibility is a particularly treacherous property, because a subtly wrong class hierarchy reads exactly like a correct one. A hallucinated superclass does not look like a hallucination; it looks like a reasonable modelling decision that a competent engineer might have made, and it will pass casual review indefinitely.

Three consequences worth planning for:

Review capacity, not generation capacity, is now the constraint. If your modelling throughput rises tenfold and your review throughput does not, the surplus becomes unreviewed model. This is Chapter 2's cognitive-budget problem arriving with a much bigger firehose.

Mechanical checking becomes more valuable, not less. Precisely because machine-generated models are fluent, the checks that do not care about fluency — is this class satisfiable, is this property declared, does this contradict its own ancestor — carry proportionally more of the load. LOG-001 catches a contradiction identically whether a human or a model introduced it, and it does not find the surrounding prose persuasive.

The decay clock speeds up. Chapter 1's decay problem is a function of change rate. Raise the change rate tenfold and an ontology that would have drifted quietly over three years drifts quietly over four months.

The rule: a model may propose. A human decides. A machine verifies. Any workflow that lets generation flow directly into an authoritative artefact has substituted fluency for correctness.

5.2 Semantics for AI: the grounding direction

The second direction is where the commercial energy currently is, and it is a better argument for semantic investment than anything in the pre-LLM era.

Agentic systems have a structural weakness: they are probabilistic components being asked to take consequential, typed actions against real systems. What they lack is not intelligence but **constraint** — a specification of what things are, how they relate, and what operations are legitimate.

That is an ontology's job description. The current framing in the field is that ontologies provide the *logical guardrails* that turn a model from a fluent text generator into a controllable actor: typed actions bound to authoritative data objects, validated before execution.

The reported effects are directionally consistent, if early:

- Grounding each individual reasoning *step* against graph-structured data, rather than only the final answer, has been reported to improve results by roughly a quarter over chain-of-thought baselines.
- Ontology-grounded retrieval reports substantially better fact recall, particularly on relational, multi-hop questions — the ones where flat vector retrieval characteristically fails, because the answer requires traversing relationships rather than matching text.

Treat specific percentages as directional — much of this literature is vendor-adjacent and the benchmarks are young. The qualitative claim is the robust one: **multi-hop, relational questions are where a graph beats a vector index, and that is exactly the question shape agents generate.**

What agents actually require from you

An agent consuming your knowledge graph needs the things SemOps produces anyway, which is a convenient alignment:

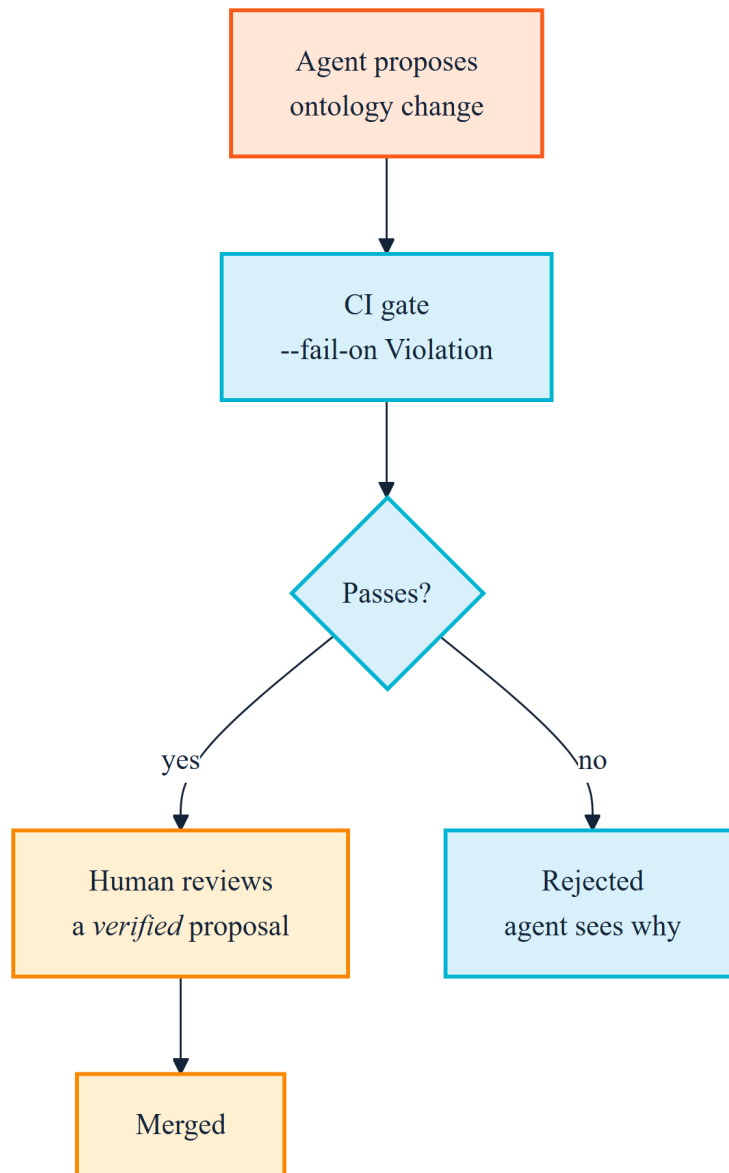
Agent requirement	SemOps artefact	Chapter
Stable identifiers that do not silently change meaning	Versioned ontology, evidence-based semver	11
Machine-readable descriptions of terms	Generated documentation, <code>rdfs:label</code> , definitions	12
A guarantee the data conforms to the model	SHACL/registry validation at ingest	10
Knowing what changed between versions	<code>version-diff</code> , migration annotations	11
Knowing who asserted what	Named graphs, provenance	3, 12

Note the second row in particular. The `qua-001` check — *a term has no `rdfs:label`* — has always been easy to dismiss as documentation hygiene. When the consumer is an agent selecting terms from your vocabulary, an unlabelled term is one the agent cannot reliably use. Label coverage has quietly become a functional requirement.

5.3 Agents as participants in the pipeline

The more interesting near-term shift is agents acting *inside* the SemOps pipeline rather than only consuming its output: opening a pull request that adds a class, drafting SHACL shapes from a specification, proposing repairs for drift, triaging a check report.

This is safe **exactly to the degree that the gate is real.**



An organisation at maturity Level 1 — no version control, no automated validation — that adds an agent to its modelling workflow has added an untraceable, high-throughput source of plausible changes to an asset with no gate. That is strictly worse than having neither.

An organisation at Level 3 — CI running linting, SHACL and reasoning checks on every change — can accept agent contributions on the same terms as human ones, because the same gate applies and the same evidence is produced. **The maturity model has become a prerequisite for safely using the technology that makes the work cheaper**, which is an argument worth making to a sponsor who wants the AI benefits without the plumbing.

Two additional practices specific to agent participation:

Record authorship provenance. Which terms were machine-proposed, which were human-authored, which were machine-proposed and human-approved? When a definition is later disputed, this is the first question, and it is unanswerable retrospectively.

Keep the confidence gate. Chapter 11's repair mechanism applies suggested fixes only at or above a confidence threshold — a 100%-confidence rename derived from an explicit migration annotation applies; a name-similarity guess does not. That pattern generalises directly to agent contributions: automate what is evidenced, escalate what is inferred.

5.4 What does not change

Worth stating plainly, because it is the part most easily lost in enthusiasm.

Meaning is still political. No model can adjudicate whether finance's or product's definition of "active customer" goes in the board pack (Chapter 2). That is an authority question. A model asked to resolve it will produce a fluent compromise nobody has agreed to, and its fluency will make the absence of agreement harder to notice.

Unknown unknowns stay unknown. A model trained on documents knows what the documents say. Margaret's undocumented knowledge about the 2019 migration is not in the corpus, and the model will not flag its absence — it will produce a confident account of the STATUS field that omits the one thing that matters.

Someone still owns it. Governance abandonment is the terminal failure mode in Chapter 1, and an agent cannot be the owner. It can do the work of an owner; it cannot hold the accountability, and the moment something goes wrong the organisation will look for a person.

The cross-boundary problems are untouched. Chapter 3's supply-chain capability asymmetry, competition-law caution and negotiated release windows are commercial and legal facts. No model changes them.

5.5 The practical position

Adopt generative tooling for what it is good at — drafting, extracting, explaining, translating, triaging — and hold the line on three things:

1. **Generation never writes directly to an authoritative artefact.** Propose, decide, verify, publish.
2. **The gate must exist before the throughput increases.** Sequence the automation before the acceleration, not after.
3. **Provenance of authorship is recorded from day one.** It cannot be reconstructed.

The pleasant irony is that a well-run SemOps practice is both the *best consumer* of generative tooling — because it can safely absorb a much higher change rate — and the *best supplier* to it, because grounded, versioned, validated, well-labelled semantics is exactly what agentic systems are short of. The discipline and the technology reinforce each other, provided the discipline arrives first.

Sources

- [Ontologies, Context Graphs, and Semantic Layers: What AI Actually Needs in 2026](#)
- [Why Ontology Matters for Agentic AI in 2026: From World Models to Governable Decisions](#)
- [Ontologies Are So Back: Why AI Agents Are Reviving the Semantic Web — Latent Space](#)
- [LLM4KGOE 2026 — Workshop on LLM-driven Knowledge Graph and Ontology Engineering](#)
- [Ontologies for the Agentic Web — WordLift](#)
- [Research Brief: Ontologies for Agentic AI \(2025–2026\) — designpattern.fyi](#)

6. Stages and stories

Part II — The SemOps frame

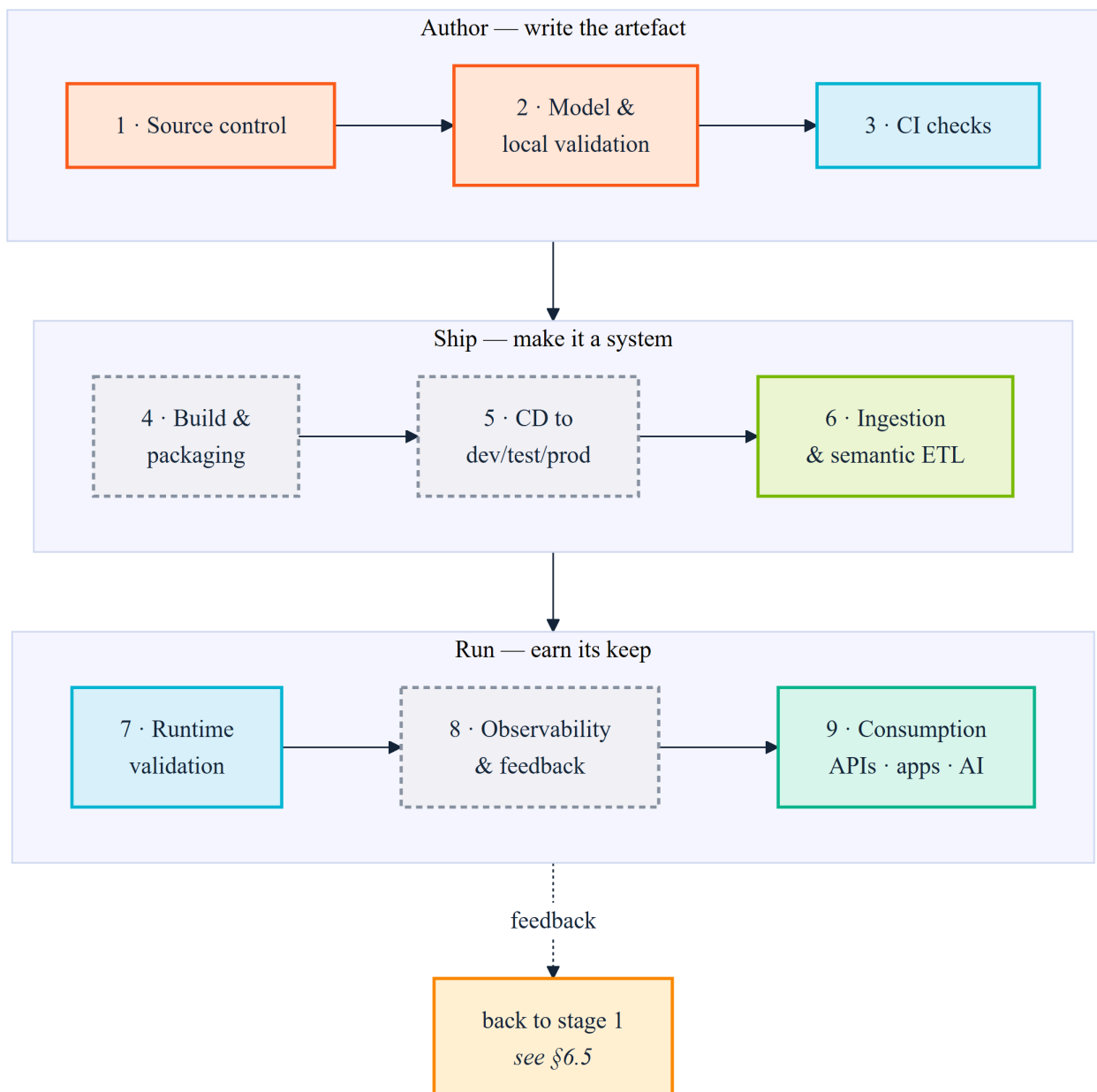
The SemOps strategy material describes the same practice three times, from three angles, and it is easy to mistake this for three different frameworks:

- a **nine-stage pipeline** — what happens to an artefact, left to right;
- a **six-layer operating model** — who is accountable, top to bottom;
- a **five-level maturity model** — how good you currently are at it.

They are one thing viewed three ways. This chapter reconciles them into a single frame, adds the **stories** — role-scoped narratives that recur throughout Part III — and states plainly which parts this manual's toolchain reaches.

6.1 The nine stages

Per the width rule in Appendix A, the pipeline is never drawn as one nine-wide row. It is three groups of three, which also happens to be how it decomposes conceptually:



The colours are load-bearing. Grey with a dashed border means **this manual's toolchain does not cover it** — stages 4, 5 and 8 are packaging, deployment and observability, and they are Kubernetes, Helm and Prometheus work rather than semantic work. Chapter 13 is the full ledger; the short version is that this toolchain owns the author and validation stages comprehensively, the ingestion stage substantially, and the deployment and observability stages not at all.

That is not a defect. Stages 4, 5 and 8 are solved problems with mature, non-semantic tooling — you deploy a triple store the way you deploy any other stateful service. The stages that have *no* good general answer, and where semantic programmes actually fail, are 2, 3, 6 and 7.

The dotted line is the whole point

Stage 9 back to stage 1 is what makes this a practice rather than a project. Without that edge you have a delivery, and Chapter 1's decay clock starts running the day it lands.

6.2 The six layers, and who is accountable

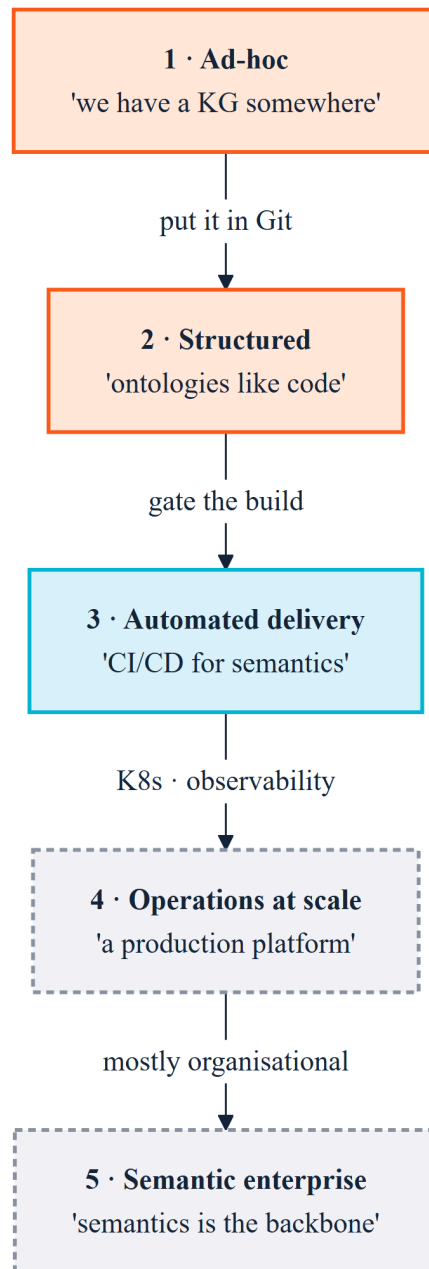
The pipeline says *what happens*. The operating model says *who answers for it*.

Layer	Purpose	Roles	Covered here?
1 • Strategy & Governance	Standards, approvals, stewardship, access policy	Semantic Product Owner, Ontology Owner / Domain Steward, Data Governance Lead, Enterprise Architect	Partly — the tooling supplies <i>evidence for</i> decisions, not the decision process (Ch. 11)
2 • Semantic Development	Ontology design, versioning, SHACL, mappings, docs	Knowledge Engineer, Ontology Engineer, Semantic Modeller, Domain Expert	Yes — Ch. 8, Ch. 12
3 • Semantic CI/CD	Automated validation, packaging, deployment	SemOps Engineer, DevOps Engineer, Automation Engineer	Validation yes, packaging/deployment no — Ch. 9
4 • Data Integration	ETL orchestration, quality checks, provenance	Data Engineer, ETL Developer, SemOps Engineer	Partly — transformation and quality yes; scheduling and orchestration no (Ch. 10)
5 • Platform & Infrastructure	Kubernetes, triple-store ops, backup, security	Platform / Cloud / Security Engineer	No — see Ch. 13
6 • Knowledge Consumption	APIs, search, dashboards, products	App Developer, Data Scientist, Business Analyst, Product Owner	Partly — checking against a live store yes; serving an API no (Ch. 12)

Layer 1 deserves a specific caution. It is the layer people assume software will provide and it is the layer software never provides. No tool approves a change. What a tool can do — and what this one does well — is ensure the human who approves it is looking at evidence rather than an opinion: *this release removes a class, therefore it is MAJOR; these three downstream queries break; one of them can be repaired automatically with 100% confidence*. The decision stays human. The **basis** for it stops being a matter of who is most confident in the meeting.

6.3 The five maturity levels

The levels answer "how good are we at this?" — and, more usefully, "what is the single next thing to fix?"



What each transition actually costs, in this toolchain:

Level	Mindset	What you would actually run	Effort to reach
1 · Ad-hoc	"We have a knowledge graph somewhere"	Nothing — this is the before-state	—
2 · Structured	"We treat ontologies like code"	ontology and checks locally, on a Git-tracked ontology; the VS Code extension's live diagnostics	Days. Mostly a decision, not a project
3 · Automated delivery	"We have CI/CD for semantic artefacts"	The same commands with <code>--fail-on Violation</code> in CI, plus <code>version-diff</code> on release	Weeks. The highest-value transition in the model

Level	Mindset	What you would actually run	Effort to reach
4 • Operations at scale	"Semantics is a production platform"	--engine native+sparql for speed, --sample N for large graphs, consistency-remote against the live store — plus K8s, Helm and observability this toolchain does not provide	Quarters , and mostly not semantic work
5 • Semantic enterprise	"Semantics is the backbone"	The version-diff → consistency → auto-repair loop run routinely; but most of Level 5 is organisational	Years , and mostly not a tooling question

Two observations that matter more than the table.

Level 2 → 3 is where the return is. It is the cheapest transition with the largest effect, because it converts "we have standards" into "the standards are enforced without anyone remembering to enforce them." Most of the value in this manual is concentrated there, which is why Chapter 14 sequences everything around reaching Level 3 and holding it.

Levels 4 and 5 are grey for a reason. Level 4 is largely platform engineering; Level 5 is largely organisational change. A team that has genuinely reached Level 3 and is frustrated that the toolchain will not carry them to Level 5 has correctly identified that the remaining problem is not a tooling problem. It is Chapter 2's problem.

The cross-cutting dimensions

The maturity model scores across eight dimensions rather than one, and the extended version of it adds a pointed question to each. They are worth reading as an assessment instrument, because a programme is rarely at one level uniformly:

Dimension	The question that actually discriminates
Ontology & model management	Are there explicit roles, and a real sign-off process?
Data integration & ETL	Are models and modelling patterns genuinely reused?
Reasoning & validation	Have you decided what to automate, what matters, and what can wait?
Deployment & infrastructure	Do services ship as versioned containers? APIs, content negotiation?
Observability & monitoring	Do you measure <i>awareness and engagement</i> , not just uptime?
Governance & access control	Who is the Product Owner? Who is the Service Owner?
Knowledge products	Is there a catalogue of them?
Culture & skills	Who is the <i>face</i> of the semantic service, and what culture do they promote?

The last one is the one nobody scores and everybody feels. A semantic capability with no visible, credible human face gets treated as infrastructure, and infrastructure gets cut in the second round of budget review.

6.4 The stories

The stages say what happens and the layers say who owns it. **Stories** say what someone is actually trying to get done, and they are how Part III is organised — each chapter answers a small set of them with real commands.

Authoring

As an ontology engineer, I want to know whether the change I just typed broke anything, without leaving the editor or waiting for CI. → Chapter 8

As a domain expert who does not read Turtle, I want to check that the model matches how the business actually works. → Chapter 8 (outline, graph view), Chapter 12 (docgen)

As a knowledge engineer, I want our own house rules enforced, not just the generic ones. → Chapter 8 (`--registry/--shapes/--sparql`)

Gating

As a SemOps engineer, I want bad semantic changes rejected automatically, and I want the rejection to be about *our* code. → Chapter 9 (`--fail-on, --own-namespace`)

As a reviewer, I want to see what a pull request actually changed semantically, not just textually. → Chapter 11 (`version-diff`)

Ingesting

As a data engineer, I want to know a transformation is wrong before I run it over four million rows. → Chapter 10 (`sketch`, then `triplify`, then `data`)

As a data steward, I want to know whether a finding is our problem or an upstream vocabulary's. → Chapter 10, Chapter 9

Releasing

As a domain steward, I need to know whether this release is breaking, and what breaks, before I approve it. → Chapter 11 (`version-diff`, `consistency`)

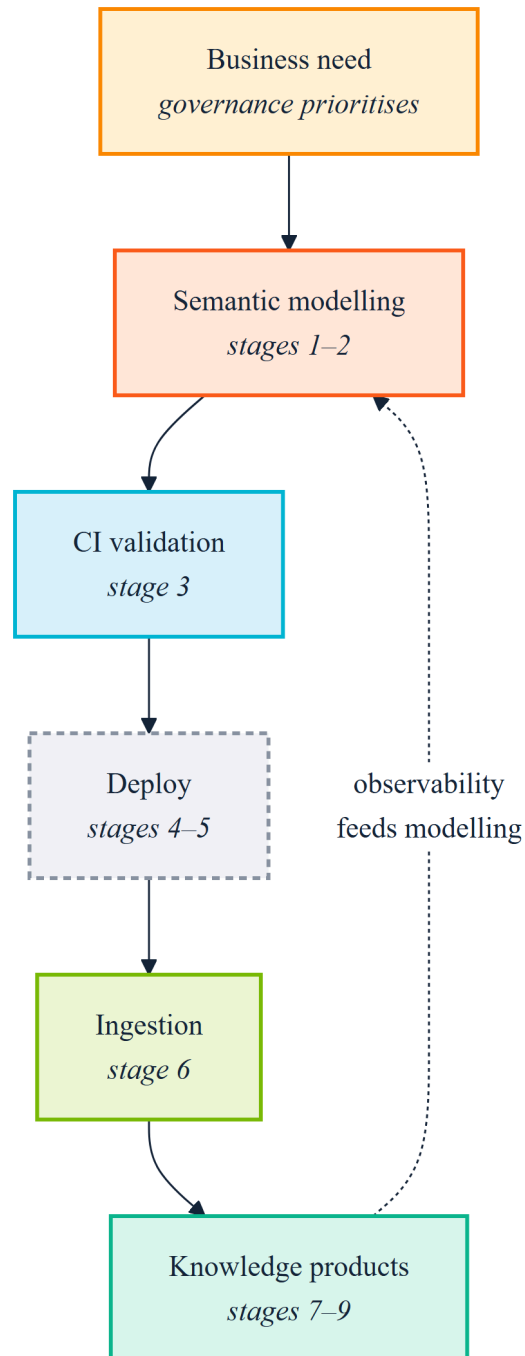
As the person who has to fix the fallout, I want the mechanical part of the migration done for me. → Chapter 11 (`--apply-repairs --min-confidence`)

Operating and consuming

As a platform owner, I want the same checks that guard the repository to run against what is actually in the triple store. → Chapter 12 (`consistency-remote`)

As an application developer or an agent, I need documentation of what the terms mean and a guarantee they will not change under me. → Chapter 12, Chapter 5

6.5 The whole frame, once



This is the continuous semantic delivery loop, and the rest of the manual is an account of how to run it with tools that exist. Chapter 7 introduces them; Part III walks the loop.

7. The toolchain

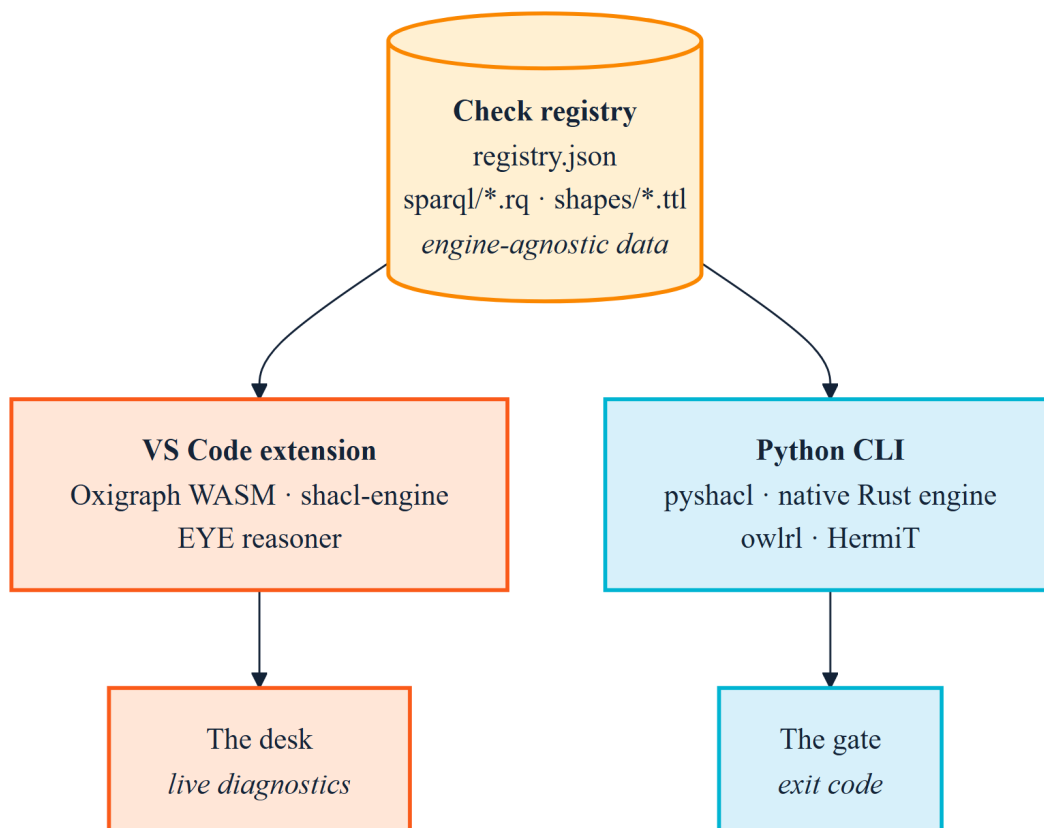
Part II — The SemOps frame

Two tools, and one architectural idea that matters more than either of them.

	Ontology Quality Suite	Ontology Development Suite
Repository	consolidated_ontology_suite_python	consolidated_ontology_suite_webapp
Form	Python CLI + importable library	VS Code extension
Runtime	Python; optional Java for DL reasoning	In-process WASM/JS — no Python or Java needed
Belongs in	CI, release control, pipelines, scheduled jobs	The author's editor, before anything is committed
SemOps stages	3, 6, 7, and the release parts of 1	1, 2
Answers	"Does this pass?"	"What did I just do?"

7.1 The idea: one registry, two runtimes

The suite's quality rules are not code. They are **data** — a `registry.json` plus a directory of SPARQL `.rq` files and SHACL `.ttl` shape files. The Python CLI reads that data and evaluates it with `pySHACL`, a native Rust engine, and `rdflib`. The VS Code extension reads *the same data*, copied into its own `resources/checks-registry/`, and evaluates it with `Oxigraph` compiled to WASM, `shacl-engine`, and the `EYE` reasoner.



The consequence is the thing to hold on to:

The rule that fails your build is the same rule that underlined the problem in your editor twenty minutes earlier.

That property is worth more than any individual feature in either tool. A team whose local linting disagrees with its CI develops a learned distrust of both — green locally, red in CI, and the gate becomes something to be worked around rather than something that helps. Sharing the rules as data rather than reimplementing them twice is what prevents that.

It also means a project-specific check written once (Chapter 8) is available to both, which is how house rules stop being a wiki page nobody reads.

7.2 Installing them

The Python suite

```
pip install ontology-quality-suite
```

or, working from a checkout:

```
uv sync                                # base install
uv sync --extra reasoner                # + owlready2 for real OWL2 DL reasoning (needs Java)
```

The reasoner extra is optional by design, and Chapter 4 explains why that design matters. Without it you still get the always-on owl closure and pattern checks, which catch the fixture's deliberate contradiction on their own — you also get an explicit REA-022 finding telling you the DL reasoner did not run and what that means for completeness. You are never silently under-checked.

Two invocation styles appear in this manual:

```
ontology-quality-suite <command> ...    # installed entry point
python -m ontology_suite <command> ...   # module form, from a checkout
```

They are equivalent. The module form is used in Part III because that is how the commands were actually run while writing it.

The VS Code extension

Install the .vsix from the webapp repository. Core functionality — authoring, live diagnostics, local checks, metrics, graph view, query workbench — has **no external runtime dependency at all**: Oxigraph, shac1-engine and eyereasoner are WASM or pure JS.

The Python CLI is detected via the ontologySuite.pythonCliPath setting or on PATH, and is used only for the two commands that genuinely need it — *Run Deep Validation* (full OWL2 DL reasoning) and *Run Full Triplify* (the real oxi-gen binary). Both degrade gracefully when it is absent.

That zero-dependency property is a governance fact, not just a convenience. As Chapter 3 argues, a validation step that requires a JVM excludes a class of supply-chain partners from self-service compliance.

7.3 Decision table: question → command

You want to...	Run	Chapter
Know if the ontology itself is sound — no data	ontology	8
Run the full registry against ontology and/or data	checks	8, 9
See only findings in terms you own	checks --own-namespace <IRI prefix>	9
Check CSV→RDF queries against the ontology, no CSV needed	sketch	10
Actually produce RDF from CSV	triplify	10
Assess real triplified data	data	10
Compare two versions, get MAJOR/MINOR/PATCH	version-diff	11

You want to...	Run	Chapter
Find and auto-repair ontology↔query drift	consistency	11
Check a controlled-vocabulary layer too	pattern-consistency	11
Run the checks against a live triplestore	consistency-remote	12
Produce something a stakeholder can read	docgen	12
Run everything applicable, one report	run	9

And the editor-side equivalents, by VS Code command:

You want to...	Command
Start a new ontology with imports wired up	<i>New Ontology</i>
Add a class in the right place in the hierarchy	<i>Add Class / Add Subclass / Add Sibling Class</i>
Run the registry locally, in-process	<i>Run Local Checks</i>
See the model as a picture	<i>Visualize Subject Graph</i>
Draft an ontology and query from a raw CSV	<i>Infer Ontology + Query from CSV</i>
Preview a transformation against sample rows	<i>Query Workbench</i>
See DL expressivity and OWL2 profile membership	<i>Show Metrics & DL Expressivity</i>
Escalate to full OWL2 DL reasoning	<i>Run Deep Validation</i>

7.4 Two flags worth knowing before you start

--engine

checks, data and run accept --engine. It defaults to native+sparql when the optional Rust SHACL engine is installed, and both (pyshacl) otherwise. This is not only about speed, though the speed difference is large — the suite's architecture documentation records benchmarks up to roughly 665× on a real-sized ontology.

It is also about correctness. **pySHACL has a confirmed bug: it ignores sh:severity declared inside sh:sparql SPARQL-based constraints, reporting violation regardless of what the shape actually says.** The native engine handles this correctly. If you have shapes that deliberately declare warning severity inside SPARQL constraints and your CI is failing on them, this is why.

--verbose

Every subcommand takes -v/--verbose, which prints what each input option actually resolved to *before* running: which files matched a glob, which owl:imports resolved and from where, which engine ran.

Reach for it whenever a result looks surprising, before assuming a bug. Most "the tool is wrong" reports turn out to be an owl:imports that silently did not resolve, and --verbose shows that in one line. There is a worked instance of exactly this class of mistake in Chapter 9, where an IRI prefix off by one character produced a confidently empty report.

7.5 The fixture used throughout Part III

Every command in Part III runs against examples/acme_robotics/ in the Python suite — a small org chart built on two real, standards-track vocabularies rather than an invented domain:

Vocabulary	Used for
W3C Organization Ontology (org:)	org:OrganizationalUnit as the base class for departments
FOAF (foaf:)	foaf:Person as the base for employees; foaf:name, foaf:mbox

The fixture is **deliberately imperfect**, with each flaw commented with the check it demonstrates:

Term	Flaw	Check it triggers
acme:Contractor	rdfs:subClassOf acme:Employee and owl:disjointWith acme:Employee	LOG-001 — logically unsatisfiable
acme:hasSkill	no rdfs:label	QUA-001
acme:reports_to	local name is not lowerCamelCase; no domain or range	STY-002, STR-003

acme-org-v2.ttl is a plausible next release: it renames acme:Engineer to acme:SoftwareEngineer — a breaking change, *with* a migration annotation — and adds one genuinely additive class and property. It deliberately leaves the flaws above untouched, because real releases do not fix everything at once.

Using real external vocabularies rather than toy ones is what makes Chapters 9 and 10 possible: the most valuable lessons in this manual come from the collision between a general-purpose checker and the conventions real vocabularies actually use.

7.6 Where the deeper documentation lives

This manual is the SemOps layer. For tool-level depth, the suite's own docs:

Document	Covers
PRIMER.md	Task-oriented guide, worked examples, CI wiring, adoption path
CHECKS.md	All 50 checks, by category
ARCHITECTURE.md	Engine comparison, benchmarks, file loading
REASONING.md	Reasoner backends and their real limitations
CONSISTENCY_AND_REPAIR.md	Finding-kind → fix-kind → confidence table
EXTENDING.md	Authoring your own checks
FUSEKI.md	Live-triplestore manifest spec and Python API
VERSIONING.md	Semver rules for ontologies
UPDATING.md	Sequencing and rollback for coordinated rollouts
ACME_ROBOTICS_WALKTHROUGH.md	The fixture, end to end, plus two runnable notebooks

The webapp repository's README.md and TUTORIAL.md do the same for the extension.

8. Model and validate

Part III — The practice · SemOps stage 2 · Operating-model layer 2

Stories answered here *As an ontology engineer, I want to know whether the change I just typed broke anything, without leaving the editor. As a domain expert who does not read Turtle, I want to check the model matches how the business actually works. As a knowledge engineer, I want our own house rules enforced, not just generic ones.*

Stage 2 is where the artefact is written, and it is the stage with the highest leverage per pound spent. A problem caught here costs a keystroke. The same problem caught in CI costs a context switch; caught in production it costs an incident and a meeting.

8.1 The inner loop belongs in the editor

The SemOps pipeline blueprint lists "modelling and local validation" as a stage in its own right, before CI, and this is not a formality. If the only way to find out whether a model is sound is to commit and wait, the feedback loop is minutes long, and a minutes-long loop changes how people work: they batch changes, they commit less often, and they stop experimenting.

The Ontology Development Suite exists to make that loop seconds long. Everything below runs **in-process**, via WASM, with no Python or Java involved.

Structure you can see

The Ontology Outline renders classes and properties as a nested, Protégé-style hierarchy in the Explorer sidebar — object and datatype properties kept as separate subtrees, rather than one flat list:

- ▼ Classes (4)
 - ▼ Employee
 - Contractor
 - Engineer
 - OrganizationalUnit
- ▼ Object Properties (2)
 - works in
 - reports to
- ▼ Datatype Properties (3)
 - has skill
 - has employee ID

This is the artefact from Chapter 2 that a domain expert can actually validate. Nobody needs to read an axiom to say "*Contractor should not be under Employee — contractors are not on our payroll.*" The priesthood problem is defeated by rendering, not by training.

Authoring happens in the same tree: right-click any class or property node for *Add Subclass*, *Add Sibling Class* (which inherits the sibling's own parents) or *Add Sub-property*. Dragging one class onto another adds an **additional** parent — deliberately never a destructive move, and the confirmation message says so.

Live diagnostics, including one SHACL cannot give you

Run Local Checks evaluates the registry's 39 SPARQL and 6 SHACL-SPARQL checks, plus OWL2-RL-style inference and contradiction detection, into the standard Problems panel.

One check is worth calling out because it addresses a genuine blind spot in SHACL itself. **voc-001, the closed-world vocabulary check**, catches the typo:

```
:bob a acme:Empolyee .      # note the transposition
```

SHACL's open-world semantics will never flag this. Nothing *contradicts* the existence of an undeclared class — it is simply never asserted to. The graph is valid; it is just about something that does not exist.

voc-001 walks every triple's predicate and, for a fixed set of term-referencing predicates (`rdf:type`, `rdfs:subClassOf`, `rdfs:domain`, `rdfs:range`, `owl:equivalentClass`, `sh:targetClass`, and others), its object too — flagging any IRI not declared in the document or its resolved imports.

Critically, it is **scoped to namespaces the graph has some closed-world knowledge of**: at least one term must already be declared there. An external vocabulary that was never actually imported is left alone rather than flooded with false positives, and `rdf:/rdfs:/owl:/sh:/skos:/xsd:` are excluded automatically. That scoping decision is the same instinct as Chapter 9's central lesson: **a check is only useful if its findings are about things you control**.

Quick Fix, with the diff shown first

Sixteen checks offer a one-click lightbulb repair, each computed from a real SPARQL Update and completed where relevant by the project's own `.ontology-suite/standards.json`. Nothing is ever written without an explicit confirmation showing the exact triples that will change.

That confirmation step is not timidity. An auto-fix that silently rewrites an ontology teaches the team not to trust the tool, and one bad silent rewrite costs more confidence than fifty good ones earn.

Metrics and expressivity, live

Show Metrics & DL Expressivity reports OntoQA-style schema metrics — class and property counts, inheritance and relationship richness, hierarchy depth — plus a DL-expressivity label such as `ALCHIQ(D)` and OWL2 EL/QL/RL profile-membership badges. The expressivity label also sits in the status bar.

The ontology is never *restricted* to a profile. The display just shows where it currently sits, which lets you develop deliberately in a lighter profile when a downstream consumer needs one — and, more usefully, notice the moment a casual edit pushes you out of it.

Seeing what the reasoner adds

Visualize Subject Graph renders a subject neighbourhood as a real SVG via Graphviz compiled to WASM. Its most instructive toggle is **Show inferred (reasoner closure)**, which runs the EYE reasoner over the neighbourhood and overlays derived triples as dashed edges alongside the asserted graph's solid ones.

For anyone learning what an ontology actually *does*, this is the single most valuable view in either tool: the difference between what you wrote and what follows from what you wrote, drawn on one picture. It is also the fastest way to discover that a subclass chain you thought was harmless entails something you did not intend.

Competency questions as tests

`.cq.rq` files — SPARQL ASK/SELECT with expected-result directives — appear in the VS Code Test Explorer and run like unit tests.

This is worth more than it first appears. A competency question is the closest thing ontology work has to a requirement: *"can this model answer 'which contractors report to someone in Engineering?'"* Making them executable turns a modelling discussion into a regression suite, which is exactly the move Chapter 2 recommends for surviving maintenance fatigue — the expert's requirement, captured once, checked forever.

8.2 The tightest CLI loop: ontology

When you want the same judgement from a terminal — or from a script — the narrowest command is `ontology: no data`, just *"is the schema itself sound?"*

```
python -m ontology_suite ontology \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --out-dir out/ontology
```

Real output from that command against the fixture:

```
Findings: 2 total (1 Violation, 0 Warning, 1 Info)
Reports written to: out/ontology
- out/ontology/report.html (start here)
- out/ontology/full_results.csv
```

The two findings:

ID	Severity	What it found
LOG-001	Violation	acme:Contractor is disjoint with its own transitive superclass acme:Employee, making it unsatisfiable
REA-022	Info	The external DL reasoner could not run; only the always-on owlrl closure and pattern checks executed

LOG-001 is the deliberate contradiction in the fixture, caught without any external reasoner. Its message is worth reading in full, because it is a model of what a check message should be:

Class `https://acme.example.org/ns/Contractor` is asserted disjoint with its own (transitive) superclass `https://acme.example.org/ns/Employee`, which makes `https://acme.example.org/ns/Contractor` logically unsatisfiable.

Remediation: Remove either the subclass axiom or the disjointness axiom; the two together are contradictory.

No description-logic background is required to act on that.

REA-022 is Chapter 4's worked example of consuming research-grade components responsibly: the DL reasoner failed, and rather than crashing or — far worse — silently reporting success, the suite emitted a first-class finding naming exactly which checks did not run and what that costs you in completeness.

Want a specific OWL2 profile checked? `--profile EL` (repeatable). It is off by default; ask for it explicitly when a downstream consumer needs a lighter profile.

8.3 The full registry: checks

`ontology` is the schema-soundness subset. `checks` runs the whole 50-check catalogue across eight categories — structural integrity, logical cogency, naming style, documentation, efficiency, data quality and more — against an ontology, a data graph, or both.

```
python -m ontology_suite checks \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --engine sparql \
  --out-dir out/checks --fail-on never
```

Run exactly like that, against the fixture with its real org: and FOAF imports resolved, this reports **296 findings** — 50 Violations, 162 Warnings, 84 Infos.

Four or five of those are Acme's. The rest are the registry doing its job thoroughly against the internals of the W3C Organization Ontology and FOAF.

That number is the subject of Chapter 9, and it is the most important practical lesson in this manual. Do not wire this command into CI as written.

8.4 Your own rules, without forking anything

SemOps layer 2 calls for SHACL constraint modelling as an ongoing knowledge- engineering activity, and layer 1 lists "modelling standards" as a core artefact. Both are hollow unless a project's own rules can be enforced as easily as the built-in ones.

The registry is data, so this needs no code. Point the CLI at your own directory:

```
python -m ontology_suite checks \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --data out/triplify/employees.ttl \
  --registry examples/acme_robotics/custom_checks/registry.json \
  --sparql examples/acme_robotics/custom_checks/sparql \
  --engine sparql --out-dir out/acm001
```

The fixture ships exactly one such check, ACM-001, which encodes an Acme HR requirement no general-purpose registry could know about — every employee must carry a traceable employee ID:

```
CONSTRUCT {
  _:r a sh:ValidationResult ;
  sh:resultSeverity sh:Violation ;
  sh:focusNode ?this ;
  sh:resultMessage "Employee {$this} has no acme:hasEmployeeId." ;
  sh:sourceConstraintComponent acme:ACM-001 .
}
WHERE {
  ?this a acme:Employee .
  FILTER NOT EXISTS { ?this acme:hasEmployeeId ?id }
}
```

with a registry entry giving it a title, a category, a default severity, a description and — notably — a **remediation** string:

```
{
  "id": "ACM-001",
  "category": "structural",
  "default_severity": "Violation",
  "title": "Employee missing an employee ID",
  "description": "An acme:Employee individual has no acme:hasEmployeeId value -- every employee record must be traceable to Acme's HR system of record.",
  "remediation": "Add acme:hasEmployeeId \"ACM-NNNN\" to the employee, or fix the triplify query/CSV mapping that should have set it."
}
```

Three things about this pattern are worth internalising.

Checks are discovered by walking the directory, not from a manifest list. To remove a check, delete its .rq file; to add one, drop in a file and a registry entry. This makes a project's rule set a normal, reviewable part of the repository.

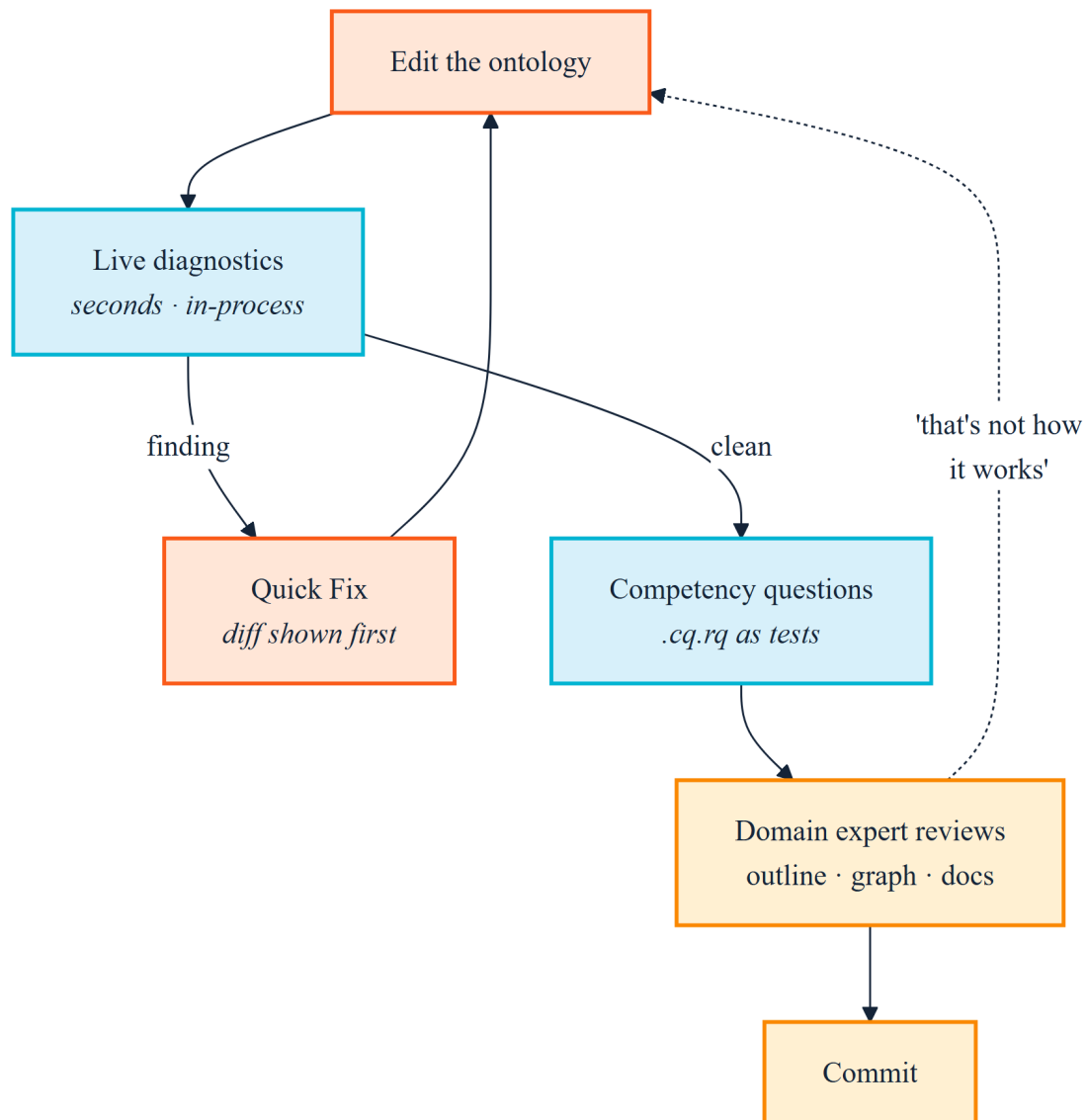
A minimal registry replaces the defaults. The fixture's registry contains only ACM-001, so running as above runs *only* that check. To keep the built-in 50 running alongside, point --shapes/--sparql at the installed resources directory as well. Find it with:

```
python -c "from ontology_suite import config; print(config.PACKAGE_RESOURCES)"
```

The remediation field is the deliverable. A rule that says "ACM-001 failed" generates a ticket. A rule that says "add acme:hasEmployeeId "ACM-NNNN" to the employee, or fix the triplify query that should have set it" generates a fix. When you write house rules, spend the time on that field — it is where the knowledge actually lives, and it is what makes the check legible to someone who was not in the room when the rule was agreed.

This is also the artefact Chapter 3 recommends publishing to supply-chain partners. A registry directory is a distributable thing: a partner can run your rules before submitting, instead of discovering them through rejection.

8.5 The loop, drawn



The dotted edge is the one that matters most and is most often missing. A modelling process where the domain expert's correction has no path back into the artefact is the extractive workshop from Chapter 2 wearing a process diagram.

8.6 Maturity checkpoint

Running the commands in this chapter, on a Git-tracked ontology, with the extension installed on engineers' machines, puts a team at **maturity Level 2 — Structured Semantic Development**: ontologies treated like code, repeatable modelling, early governance, reduced risk of breaking changes.

Level 2 is a decision more than a project. What it does *not* yet give you is enforcement: nothing here stops someone committing an ontology that fails every check. That is Chapter 9, and it is the transition worth spending real effort on.

9. Continuous integration

Part III — The practice · SemOps stage 3 · Operating-model layer 3

Stories answered here *As a SemOps engineer, I want bad semantic changes rejected automatically, and I want the rejection to be about our code.*

Stage 3 is where SemOps stops being a philosophy. The blueprint's stated goal is blunt — “*automatically reject bad semantic changes*” — and the mechanism is almost embarrassingly simple: one flag, `--fail-on`, and a non-zero exit code.

The simple part takes a paragraph. The rest of this chapter is about the part that actually determines whether the gate survives its first month, which is **what the gate reports**.

9.1 The mechanism

```
python -m ontology_suite ontology \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --fail-on Violation
```

`--fail-on` takes Violation, Warning, Info or never. The command exits non-zero when a finding at or above that severity exists. Against the fixture this exits non-zero, because LOG-001 — the deliberate Contractor/Employee contradiction — is a real Violation.

That is the whole gate. `version-diff` has its own variant (`--fail-on major`), and consistency and pattern-consistency use `--fail-on-misalignment` and `--fail-on-mismatch`; nothing more exotic is needed for a green/red build.

9.2 The problem: 296 findings

Now run the full registry the obvious way — the whole 50-check catalogue against the ontology, with its real org: and FOAF imports resolved:

```
python -m ontology_suite checks \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --engine sparql --out-dir out/checks --fail-on never
```

Findings: 296 total (50 Violation, 162 Warning, 84 Info)

Every one of those findings is real. The suite is genuinely checking every triple in the merged graph — and the merged graph includes the entire W3C Organization Ontology and the entire FOAF vocabulary, complete with their own internal documentation habits, blank-node axiom style and naming choices.

Wire that into CI with `--fail-on Violation` and you have gated your build on fifty violations, of which approximately zero are yours. What happens next is predictable and happens every time: the team triages it once, concludes the tool is noisy, and either switches the gate off or adds `|| true`. The four findings that were genuinely theirs are lost with the other 292.

This is the single most common way a semantic quality gate dies. Not because the checks are wrong — they are not — but because Chapter 2's cognitive budget was never considered. A gate nobody triages is not a gate.

So: whose problems are these? A domain steward owns the correctness of *their own* additions to a shared vocabulary. They do not own the W3C's internal SHACL cleanliness, and cannot fix it.

9.3 The obvious fix is wrong

The instinctive move is to stop resolving the imports at all:

```
python -m ontology_suite checks \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --exclude-imports \
  --engine sparql --out-dir out/checks_excl --fail-on never
```

296 findings become **9**. Enormously better, and still wrong. Here is the full breakdown:

```
2 x DAT-002 Warning Dangling IRI reference
1 x LOG-001 Violation Class disjoint with its own ancestor
1 x QUA-001 Warning (missing label)
3 x QUA-004 Warning Resource missing skos:prefLabel
1 x STR-003 Warning (missing domain/range)
1 x STY-002 Warning (naming style)
```

Inspect the two DAT-002 findings and the cause is immediate:

```
focus = acme:1.0.0      path = owl:imports      value = http://xmlns.com/foaf/0.1/
focus = acme:Employee   path = rdfs:subClassOf   value = http://xmlns.com/foaf/0.1/Person
```

Both point at FOAF. `acme:Employee` is a subclass of `foaf:Person`, and with imports excluded, FOAF is not loaded, so `foaf:Person` looks like a dangling reference. It is not dangling; it is simply absent. Two of the three QUA-004 findings are the same artefact — they are against `foaf:` itself and `foaf:Person`, terms whose labels live upstream in a file you just declined to read.

--exclude-imports traded 291 irrelevant findings for 4 false ones. In some ways that is a worse failure: irrelevant findings get ignored, but false findings get *investigated*, and an engineer who spends an afternoon proving that `acme:Employee` is fine learns exactly the same lesson about the tool's trustworthiness.

9.4 The right fix: --own-namespace

Keep the imports resolved — so upstream context is real, `foaf:Person` genuinely exists, and cross-vocabulary reasoning works — and filter the **report** to findings whose focus node lies in your own namespace:

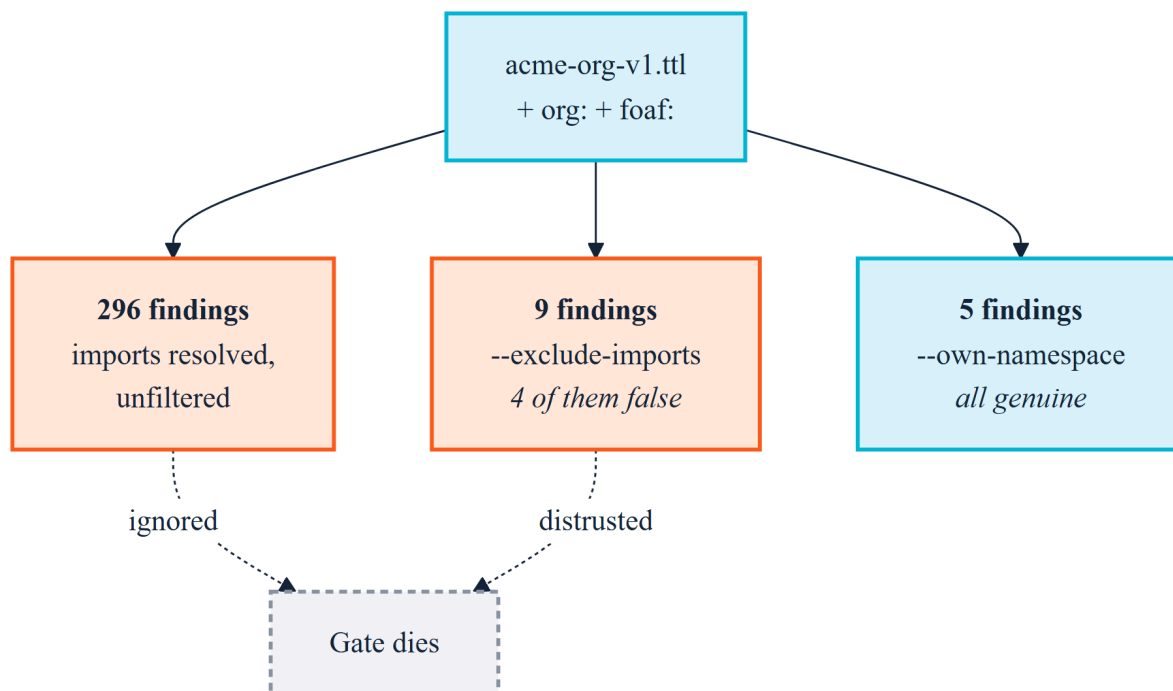
```
python -m ontology_suite checks \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --engine sparql \
  --own-namespace "https://acme.example.org/ns/" \
  --out-dir out/checks_own --fail-on never

1 x LOG-001 Violation Class disjoint with its own ancestor
1 x QUA-001 Warning (missing label)
1 x QUA-004 Warning Resource missing skos:prefLabel -> acme:hasSkill
1 x STR-003 Warning (missing domain/range)
1 x STY-002 Warning (naming style)
```

Five findings. All five are Acme's. None are artefacts.

Compare the three runs directly — all three are the same ontology, the same registry, the same engine:

Run	Findings	Genuinely yours	Upstream noise	False
--import-dir alone	296	5	291	0
--exclude-imports	9	5	0	4
--own-namespace	5	5	0	0



A real mistake worth repeating

The first attempt at that command, while writing this manual, used `--own-namespace "http://example.org/acme#"` and returned:

```
Findings: 0 total (0 Violation, 0 Warning, 0 Info)
```

A confidently empty report. The fixture's namespace is `https://acme.example.org/ns/` — different scheme, different host, different separator. `--own-namespace` is a **literal IRI-prefix string match**, not a prefix-name lookup, and a near-miss produces silence rather than an error.

Zero findings is exactly what a passing gate looks like. Two lessons:

1. **Copy the namespace from the ontology, never from memory.** It is the string after `@prefix`, in full, including the trailing `/` or `#`.
2. **Verify the filter with a known-failing input once**, and keep that check. A gate that cannot fail is indistinguishable from a gate that passes.

`--verbose` prints what every option actually resolved to before running, and is the first thing to reach for when a result looks surprising — including this one.

When to run the wide pass anyway

Do not discard the 296-finding run; just stop putting it on the pull-request path. Run it **periodically, or when an imported vocabulary changes** — as an audit rather than a gate. That is when "FOAF published a new version and something in it now conflicts with our assumptions" becomes findable, and it is a genuinely different question from "did this PR break anything."

Cadence	Command	Purpose
Every PR	<code>checks --own-namespace <yours> --fail-on Violation</code>	Gate
Weekly / on upstream change	<code>checks --import-dir ... --fail-on never</code>	Audit

9.5 Wiring it up

The pattern generalises to three gate types, one per kind of pull request.

```

name: semantic-ci
on: [pull_request]

jobs:
  ontology:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: uv sync

      # 1. Is the schema itself sound?
      - name: Ontology quality gate
        run: |
          uv run ontology-quality-suite ontology \
            --ontology ontology/acme-org.ttl \
            --import-dir vendor/vocab \
            --fail-on Violation

      # 2. Does it pass our rules, scoped to terms we own?
      - name: Registry checks (our namespace only)
        run: |
          uv run ontology-quality-suite checks \
            --ontology ontology/acme-org.ttl \
            --import-dir vendor/vocab \
            --own-namespace "https://acme.example.org/ns/" \
            --engine sparql \
            --fail-on Violation

      # 3. Is this release breaking, and does anything downstream break with it?
      - name: Change impact
        run: |
          uv run ontology-quality-suite version-diff \
            ontology/acme-org.ttl.base ontology/acme-org.ttl \
            --json --fail-on major
        continue-on-error: true # informational: a MAJOR bump needs sign-off, not a block

      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: semantic-reports
          path: out/

```

Four points about that job.

Vendor your imports. `--import-dir vendor/vocab` resolves `owl:imports` against local copies. `--allow-network` exists, and CI is the last place you want it: a build whose result depends on a third party's uptime is not reproducible, and an upstream vocabulary that changes silently changes your gate silently.

Upload the reports as artefacts. `report.html` is the thing a reviewer actually reads; `full_results.csv` is the thing you diff between runs. Both are written on every run regardless of exit code.

A MAJOR bump should not block the build. It should *demand a human*. The job above records it and continues; the enforcement belongs in a branch protection rule requiring a named reviewer, which is a governance control rather than a technical one. See Chapter 11.

Different PRs need different gates:

PR touches	Gate to run
The ontology	ontology, then checks <code>--own-namespace</code>
A transformation query	sketch first — it needs no CSV (Ch. 10)
A taxonomy of controlled values	pattern-consistency (Ch. 11)
Data	data, with <code>--sample N</code> if the graph is large

9.6 The one-command version

Once you know which stages apply, `run` composes them into a single merged report:

```
python -m ontology_suite run \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --queries examples/acme_robotics \
  --own-namespace "https://acme.example.org/ns/" \
  --engine sparql --fail-on Violation --out-dir out
```

Convenient for a nightly job. Less good as a PR gate, because a single merged pass/fail tells a reviewer less than three named steps that each fail for a stated reason. Prefer the explicit steps where a human reads the result.

9.7 Across the boundary

Chapter 3 noted that the gate has no jurisdiction outside your own organisation. The mechanics are unchanged; the disposition of the output is not:

	Internal	Cross-boundary
Non-zero exit	Blocks the merge	Generates a notice with a deadline
report.html	A reviewer reads it	The partner reads it
The registry directory	Internal config	A published artefact partners run themselves

That last row is the highest-leverage cross-boundary move available, and it costs nothing you have not already built: your check registry is a directory of files, and `--registry/--shapes/--sparql` means a partner can point at it and validate before submitting rather than after being rejected.

9.8 Maturity checkpoint

A team running these gates on every pull request, with reports retained, is at **maturity Level 3 — Automated Semantic Delivery** for the validation dimension: predictable releases, faster iteration, early detection of semantic regressions.

Level 3's other dimensions — automated packaging, deployment to dev/test, orchestrated ETL, ingestion monitoring — are partly Chapter 10 and partly not covered by this toolchain at all (Chapter 13).

This is the transition worth spending real effort on. It is also the one that makes agent participation safe: as Chapter 5 argues, a gate that genuinely rejects bad changes is what lets you accept proposals from a much higher-throughput source without accepting the risk that comes with it.

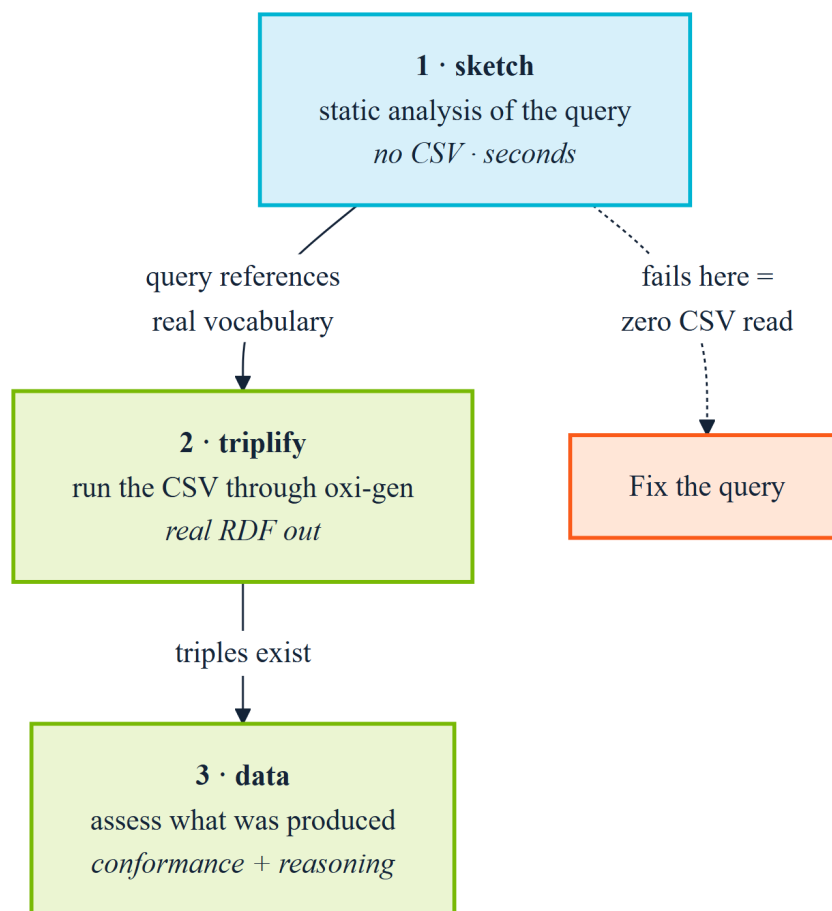
10. Ingest and transform

Part III — The practice · SemOps stage 6 · Operating-model layer 4

Stories answered here *As a data engineer, I want to know a transformation is wrong before I run it over four million rows. As a data steward, I want to know whether a finding is our problem or an upstream vocabulary's.*

SemOps layer 4 asks for ETL orchestration, incremental ingestion, data-quality checks, error handling and provenance capture. This toolchain covers the **quality and correctness** half thoroughly and the **orchestration** half not at all — there is no scheduler here, and Airflow, Argo or Prefect remain the right answers for that (Chapter 13).

What it does provide is a three-stage pipeline arranged on one principle: **cheapest check first**.



10.1 Before the pipeline: drafting from a CSV

Most real ingestion starts with a CSV somebody emailed you, and the first question is what the ontology should even look like.

The extension's *Infer Ontology + Query from CSV* profiles a raw CSV — sniffing types and cardinality — and drafts **both** a starter ontology fragment and a CONSTRUCT query wired to it. It is a starting point, not an answer: an inferred model reflects the shape of one extract, not the shape of the domain, and Chapter 2's exemplar-driven expert will find its boundary cases in minutes.

The *Query Workbench* is the more durable tool. It gives a live, debounced preview of the transformation's real triplified output against sample rows — using Oxigraph, with TARQL semantics reproduced via a standard SPARQL VALUES injection — alongside a static sketch of the CONSTRUCT template and conformance warnings against the ontology.

The point is the loop length. Discovering a wrong BIND after a full pipeline run is a twenty-minute round trip; discovering it in a live preview is immediate, and immediate feedback is what makes people willing to iterate on a transformation rather than accepting the first one that runs without erroring.

10.2 Stage 1: sketch — before any CSV is read

sketch is pure static analysis of the query text against the ontology. It answers *"does this transformation even reference vocabulary the ontology declares?"* without touching a single row of data.

```
python -m ontology_suite sketch \
  --queries examples/acme_robotics \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --import-dir examples/acme_robotics \
  --out-dir out/sketch
```

```
Findings: 35 total (0 Violation, 0 Warning, 35 Info)
Sketch used 1 query file(s); 8 triples, 7 entities.
```

The summary line is the useful part: **8 triples, 7 entities from 1 query file**. That is the shape the transformation will produce, derived from the query alone. A reviewer who expected fifteen predicates and sees eight has found a bug before any data moved.

Gotcha, hit while writing this chapter: `--queries` expects a **directory**, not a file. Passing `--queries examples/acme_robotics/employees.rq` fails with `No files matching '*.sparql,*.rq,*.tarql,*.tq' found in examples/acme_robotics/employees.rq` — the path is treated as a folder to scan. The message names the patterns it looked for, which is enough to diagnose it, but the flag reads like it takes a file. Point it at the containing folder, or narrow with `--file-pattern`.

Because sketch needs no CSV, no oxi-gen binary and no data access, it is the right gate for a pull request that changes a transformation query — it runs in seconds and needs no production data in CI.

10.3 Stage 2: triplify — produce real RDF

```
python -m ontology_suite triplify \
  --csv-dir examples/acme_robotics \
  --queries examples/acme_robotics \
  --out-dir out/triplify
```

```
Triplified 1 file(s) into out/triplify
- out/triplify/employees.ttl
```

This shells out to the real oxi-gen binary (auto-discovered, or passed with `--oxi-gen-bin`). The output is genuine RDF:

```
@prefix acme: <https://acme.example.org/ns/> .
@prefix foaf: <http://xmlns.com/foaf/0.1/> .

<https://acme.example.org/data/employee/E004> a acme:Employee , acme:Engineer ;
  foaf:mbox <mailto:margaret.hamilton@acme.example.org> ;
  foaf:name "Margaret Hamilton" ;
  acme:hasEmployeeId "E004" ;
  acme:hasRole "Test Engineer" ;
  acme:hasSkill "Automation" ;
  acme:worksIn <https://acme.example.org/data/department/QA> .
```

Useful flags in practice: `--test N` triplifies only the first *N* rows (fast iteration on a large file), `--delimiter/--tab` for non-comma input, `--no-header-row`, `--dedup N`, and `--ntriples` for line-oriented output that diffs and streams well.

`--test` deserves a habit: run it before every full run. It costs seconds and it catches the class of error where a transformation is syntactically fine and semantically wrong.

10.4 Stage 3: data — assess what was actually produced

```
python -m ontology_suite data out/triplify/employees.ttl \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
```

```
--import-dir examples/acme_robotics \  
--engine sparql --out-dir out/data --fail-on never  
Findings: 363 total (58 Violation, 185 Warning, 120 Info)
```

Three hundred and sixty-three — and Chapter 9's lesson applies identically. Scope it:

```
python -m ontology_suite data out/triplify/employees.ttl \  
--ontology examples/acme_robotics/acme-org-v1.ttl \  
--import-dir examples/acme_robotics \  
--engine sparql \  
--own-namespace "https://acme.example.org/" \  
--out-dir out/data_own --fail-on never  
Findings: 37 total (13 Violation, 22 Warning, 2 Info)
```

Note the namespace here is `https://acme.example.org/` — one level up from the ontology's `https://acme.example.org/ns/` — because the data individuals live under `https://acme.example.org/data/`. The filter is a literal IRI-prefix match, so it must cover **both** the schema terms and the instance IRIs. Choosing that prefix deliberately is part of designing your IRI scheme, and it is much easier to get right at the start than to retrofit.

What the 37 actually say

Scoped, the findings become readable, and they fall into three genuinely different kinds — a distinction worth teaching a team explicitly, because treating them identically is what makes people ignore all three:

Real data defects. The largest group:

```
STR-006 [Violation] Untyped object of a triple  
Object ../data/department/ENG of triple (../employee/E001 acme:worksIn)  
is never given a type
```

```
DAT-002 [Warning] Dangling IRI reference  
Reference target ../data/department/ENG is never used as a subject
```

Both are correct and both point at the same real gap: the transformation produces employees who `worksIn` a department IRI, but **nothing in the pipeline ever produces the departments themselves**. Every employee references a resource that does not exist in the graph. That is exactly the class of defect this stage exists to catch, and it is invisible in the CSV, invisible in the query text, and obvious the moment you assess the output.

Inherited schema problems. LOG-001, QUA-001 on `acme:hasSkill`, STR-003 and STY-002 on `acme:reports_to` — the same ontology findings from Chapter 8, reappearing because data evaluates the ontology alongside the data. Not new information; not noise either. If the schema is broken, the data built on it inherits the problem.

Coverage signals, not errors.

```
CNF-005 [Info] Ontology class never populated  
Class acme:Contractor is declared in the ontology but the data graph  
never uses it as an rdf:type
```

Nothing is wrong. `acme:Contractor` is declared and this extract contains no contractors. As an `Info` this is right, and it is genuinely useful: a class that is *never* populated across any extract is either dead vocabulary or a transformation that quietly stopped emitting it. Worth reviewing periodically; never worth failing a build on.

The reasoner, observed twice

One finding in the scoped data run is worth flagging:

```
REA-021 [Violation] Class found unsatisfiable by an external DL reasoner  
acme:Contractor is unsatisfiable (equivalent to owl:Nothing)  
according to the hermit reasoner
```

The HermiT DL reasoner ran here and independently confirmed what the pattern- based LOG-001 found by a completely different route — a rule-based closure and a real description-logic reasoner agreeing on the same contradiction. Two independent confirmations of one deliberate flaw is the ideal outcome, and it is the reason the suite runs both rather than choosing.

Worth recording honestly: in the ontology pass in Chapter 8, on the same machine and the same fixture, that same reasoner **failed to start** and emitted REA-022 instead. Same tool, same session, different code path, different outcome. No explanation is offered here because none was established — but it is a live instance of Chapter 4's point:

a research-lineage component's failure modes are yours to discover, and the thing that makes it safe to depend on is that the wrapper told you clearly which checks did not run, rather than reporting success.

Large graphs

For a data graph too large to reason over in full, `--sample N` caps the *reasoning* pass to a bounded-description sample of *N* named subjects while still running the full (cheap) registry over everything:

```
python -m ontology_suite data out/triplify/employees.ttl \
  --ontology examples/acme_robotics/acme-org-v1.ttl \
  --sample 5000 --engine sparql --out-dir out/data
```

This is the Level-4 "operations at scale" adaptation in Chapter 6: reasoning is the expensive stage, and sampling it is what keeps a nightly job inside its window.

10.5 Whose problem is it?

This is the question a data steward asks most often, and the three-way split above is the answer in general form:

Finding is about	Owner	Action
Your instance data	Data engineer	Fix the transformation or the source
Your ontology	Ontology owner	Fix the schema; it will recur on every extract
An upstream vocabulary	Nobody here	Filter it out of the gate; audit periodically
Coverage (CNF-005)	Data steward	Review, do not gate

The most expensive mistake is treating row 3 as row 1 — engineers spending days proving that a finding against someone else's vocabulary is not their fault. `--own-namespace` exists to make that impossible.

10.6 What is not here

Stated plainly, because SemOps layer 4 asks for it and this toolchain does not provide it:

- **Orchestration and scheduling.** No DAG, no retry logic, no backfill. Airflow, Argo Workflows or Prefect, calling these commands as steps.
- **Incremental ingestion.** Every run is a full pass. Incrementality is your pipeline's job.
- **Streaming.** This is batch. Kafka-shaped ingestion is out of scope.
- **Provenance capture.** The tooling does not stamp provenance onto the output. Named graphs at load time are the practical answer (Chapter 12), and it is far easier to do at ingest than to retrofit.

That last one is worth planning before your first production load, not after. Chapter 3 explains why it becomes urgent the moment data crosses an organisational boundary.

10.7 Maturity checkpoint

Running sketch as a query-PR gate, triplify with `--test` before full runs, and `data --own-namespace` on the output gives you **Level 3's data-quality dimension**: quality checks automated, regressions caught early.

Level 3 also expects orchestrated ETL and ingestion monitoring, which are not here. Level 4 expects continuous ingestion with error handling and retry logic — also not here. Those are real gaps, not omissions from this chapter; see Chapter 13.

11. Release and change

Part III — The practice · SemOps stages 1 and 4 · Operating-model layers 1 and 2

Stories answered here *As a domain steward, I need to know whether this release is breaking, and what breaks, before I approve it. As the person who has to fix the fallout, I want the mechanical part of the migration done for me.*

The SemOps overview puts Knowledge Lifecycle Management first and calls it "*the backbone of semantic governance*" — versioning, schema evolution, change-impact analysis, approval workflows. This chapter is the most directly governance-facing in the manual, and it is where the toolchain does something genuinely unusual: it turns "*is this a breaking change?*" from an opinion into a command.

11.1 Evidence-based semver: version-diff

```
python -m ontology_suite version-diff \
  examples/acme_robotics/acme-org-v1.ttl \
  examples/acme_robotics/acme-org-v2.ttl \
  --exclude-imports --out-dir out/version-diff --json
```

Real output:

Removed classes [MAJOR]:

- acme:Engineer

Removed subclass edges [MAJOR]:

- acme:Engineer no longer rdfs:subClassOf acme:Employee

Added classes [minor]:

- acme:ProductManager
- acme:SoftwareEngineer

Added properties [minor]:

- acme:hireDate

Added subclass edges [minor]:

- acme:ProductManager rdfs:subClassOf acme:Employee
- acme:SoftwareEngineer rdfs:subClassOf acme:Employee

Added equivalentClass axioms [minor]:

- acme:Engineer equivalentClass acme:SoftwareEngineer

Suggested version bump: MAJOR

Note the argument order: **old first, then new**. Reversed, you get a diff describing the inverse release.

The verdict is derived from what happened to the axioms, not from what the author believed they were doing. One removed class makes it MAJOR regardless of the seven additive changes surrounding it, and regardless of anyone's view that "nobody was using Engineer anyway."

That property is what makes it useful as a governance instrument. The release conversation stops being "*do you think this is breaking?*" — a question where the most confident person in the meeting tends to win — and becomes "*the diff says MAJOR; who signs off?*"

`--fail-on major` makes it enforceable, and `--json` writes `diff.json` alongside the human-readable `diff.txt`, which is what Chapter 3 recommends attaching to a partner-facing change notice so downstream tooling can act on it without parsing prose.

11.2 Change impact: consistency

Knowing a release is breaking is half the answer. *What* breaks is the other half.

The fixture's transformation query types every employee `acme:Engineer` — the exact term v2 renames.

```
python -m ontology_suite consistency \
  --new examples/acme_robotics/acme-org-v2.ttl \
  --old examples/acme_robotics/acme-org-v1.ttl \
  --queries examples/acme_robotics \
  --import-dir examples/acme_robotics \
  --out-dir out/consistency
```

consistency runs the version diff *and* checks every transformation query against the new ontology:

Suggested version bump: MAJOR

```
1 class(es) used in TARQL but not declared in the ontology set:
  [undeclared_class] acme:Engineer
```

```
1 suggested repair(s):
  [rename_iri] examples\acme_robotics\employees.rq (confidence 100%)
    Update to the renamed ontology term(s): acme:Engineer -> acme:SoftwareEngineer
```

Wrote 1/1 suggested repair(s) as .patch files under out/consistency/repairs (confidence >= 50%). Re-run with --apply-repairs to apply them directly.

This is SemOps' "change-impact analysis" and "approval workflows for model changes" made concrete. The steward approving this release is not being asked to imagine consequences; they are looking at the affected file, the affected term, and a proposed fix with a confidence score.

By default nothing is modified — repairs are written as reviewable .patch files under --out-dir/repairs.

11.3 The migration annotation, and its direction

The 100% confidence above is not pattern-matching on names. It comes from an explicit migration annotation in v2:

```
acme:Engineer owl:equivalentClass acme:SoftwareEngineer .
```

The gotcha that costs an afternoon: rename detection recognises this annotation in **one direction only** — asserted *from* the old, retiring IRI *to* the new one. owl:equivalentClass is logically symmetric, and any reasoner treats both directions identically, but writing it the other way round — acme:SoftwareEngineer owl:equivalentClass acme:Engineer, arguably the more natural way to write "here is the new term and what it replaces" — silently drops confidence to name-similarity guessing.

The failure is silent, which is what makes it expensive: you get a lower confidence score and a repair that may fall below your threshold, with nothing saying why.

Always assert the migration annotation from the retiring IRI. Make it a review checklist item. Across an organisational boundary (Chapter 3) it is the difference between a partner's tooling identifying the replacement automatically and a human reading your release notes.

11.4 Applying repairs

```
python -m ontology_suite consistency \
  --new acme-org-v2.ttl --old acme-org-v1.ttl \
  --queries . --import-dir . \
  --apply-repairs --min-confidence 0.7 \
  --out-dir out/consistency
```

Applied 1/1 suggested repair(s) directly to their target files (confidence >= 70%).

Verified against a scratch copy. Before:

```
?employee a acme:Employee, acme:Engineer ;
```

After:

```
?employee a acme:Employee, acme:SoftwareEngineer ;
```

The confidence gate is the safety mechanism: a 100%-confidence rename backed by an explicit annotation applies at a 0.7 threshold; a name-similarity guess does not. Automate what is evidenced; escalate what is inferred.

A caveat found by running it

The repair is a **textual substitution across the whole file, including comments**. The fixture's query carries an explanatory comment:

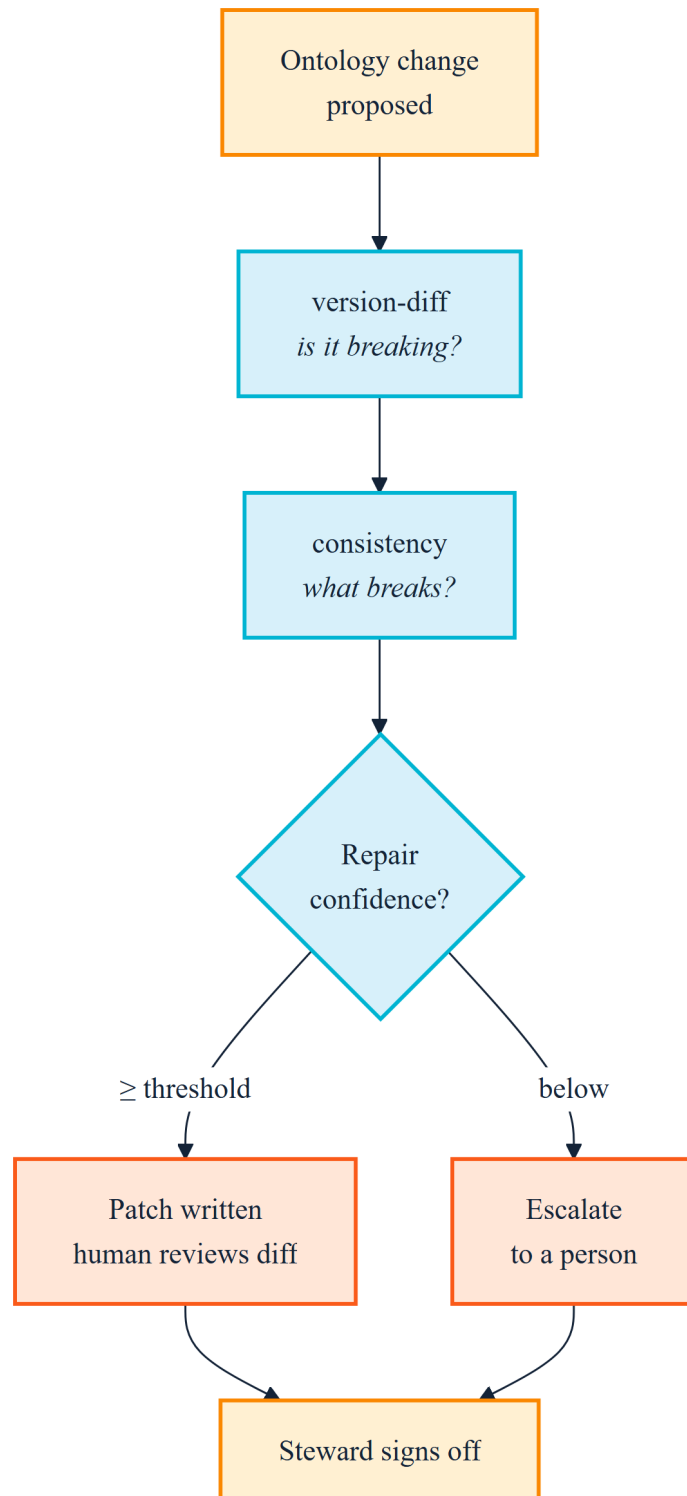
```
# - every row is typed acme:Engineer, which v2.0.0 renames to  
#   acme:SoftwareEngineer (with a migration annotation) -- this query is
```

After `--apply-repairs`, that comment reads:

```
# - every row is typed acme:SoftwareEngineer, which v2.0.0 renames to  
#   acme:SoftwareEngineer (with a migration annotation) -- this query is
```

Now nonsense. Functionally harmless — comments do not execute — but it is a real demonstration that the repair does not distinguish code from prose.

The practical consequence: prefer the default dry-run in automation, review the `.patch` files, and apply. `--apply-repairs` is excellent for a developer working locally who will read the diff before committing; it is a poor fit for an unattended CI job that commits its own output.



11.5 The taxonomy layer: pattern-consistency

consistency has a documented blind spot: it has no notion of a **taxonomy** — a controlled vocabulary of permitted values sitting on top of the ontology. pattern-consistency checks that boundary and the three around it together: ontology ↔ taxonomy ↔ transformation ↔ output data.

The fixture is built to exercise exactly this. `taxonomy.tt1` declares three valid departments as SKOS concepts:

```

dept:ENG a acme:Department, skos:Concept ;
  skos:inScheme acme:DepartmentTaxonomy ;
  skos:prefLabel "Engineering"@en .

```

ENG, QA and SALES. But `employees.csv` contains a row whose department is MKT.

```
python -m ontology_suite pattern-consistency \
--queries examples/acme_robotics \
--ontology examples/acme_robotics/acme-org-v1.ttl \
--ontology examples/acme_robotics/reference_vocab/foaf.rdf \
--taxonomy examples/acme_robotics/taxonomy.ttl \
--output-data out/triplify/employees.ttl \
--out-dir out/pattern-consistency

== taxonomy <-> output data ==
[undeclared_taxonomy_reference] ../data/department/MKT is used as the
value of acme:worksIn in the data graph but is not declared as an
individual anywhere in the given taxonomy set.
```

Caught. And the mechanism matters, because it is not the obvious one.

The transformation does not hard-code MKT anywhere. The department IRI is built dynamically, per CSV row:

```
BIND(IRI(CONCAT(..., ?department)) AS ?dept)
```

There is no fixed literal in the query text for a text-level check to flag. The taxonomy↔transformation check — which looks for values hard-coded in the query template — cannot see this, and correctly reports nothing. It is the `--output-data` layer that catches it, by comparing the values that actually appeared in the produced graph against the declared taxonomy.

This is worth dwelling on as a general principle:

A value that varies per data row cannot be validated by reading the transformation. It can only be validated by inspecting the output. Any pipeline whose controlled values come from data rather than from the query text needs the output-data layer in its gate, not just the static checks.

Omit `--output-data` and this run reports clean — correctly, and uselessly.

Note the two `--ontology` arguments. `foaf.rdf` is passed explicitly because the transformation uses `foaf:name` and `foaf:mbox`; without it, both are reported as undeclared properties. `--ontology` is repeatable, and `pattern-consistency` requires `--queries`, `--ontology` and `--taxonomy` — `--output-data` is the optional one, and the one that does the most work.

11.6 Governance: what the tooling does and does not do

SemOps layer 1 wants approval workflows, stewardship roles and audit trails. **None of that is implemented here, and no tool should claim otherwise.** What the toolchain provides is the *evidence an approval decision needs*:

Governance question	Command	Evidence produced
Does this PR introduce a new problem in code we own?	<code>checks --own-namespace --fail-on Violation</code>	Pass/fail + findings scoped to your terms
Is this breaking, and does it need sign-off?	<code>version-diff --fail-on major</code>	MAJOR/minor/patch + the axioms behind it
What exactly breaks downstream?	<code>consistency</code>	Affected files, terms, and proposed repairs
Can any of it fix itself?	<code>consistency --apply-repairs --min-confidence</code>	Confidence-gated patches
Do controlled values still hold?	<code>pattern-consistency --output-data</code>	Values in data absent from the taxonomy

The approval *workflow* — a branch protection rule requiring a named reviewer when `version-diff` reports MAJOR — is a policy layered on top. That is the correct division: the tool establishes facts, the organisation decides what to

do about them. A tool that tried to encode the decision would be wrong about it within a quarter, because Chapter 2's territorial dynamics change faster than any configuration file.

For sequencing and rollback across an ontology/taxonomy/data rollout, the suite's own `UPDATING.md` is the closest existing document to a governance playbook.

11.7 Maturity checkpoint

`version-diff` on every release, consistency before merging a breaking change, and `pattern-consistency --output-data` where controlled vocabularies exist, covers Level 3's change-management dimension and reaches into **Level 5's "continuous semantic improvement loops"** — the `diff` → `impact` → `repair` cycle, run routinely, is the closest this toolchain gets to Level 5.

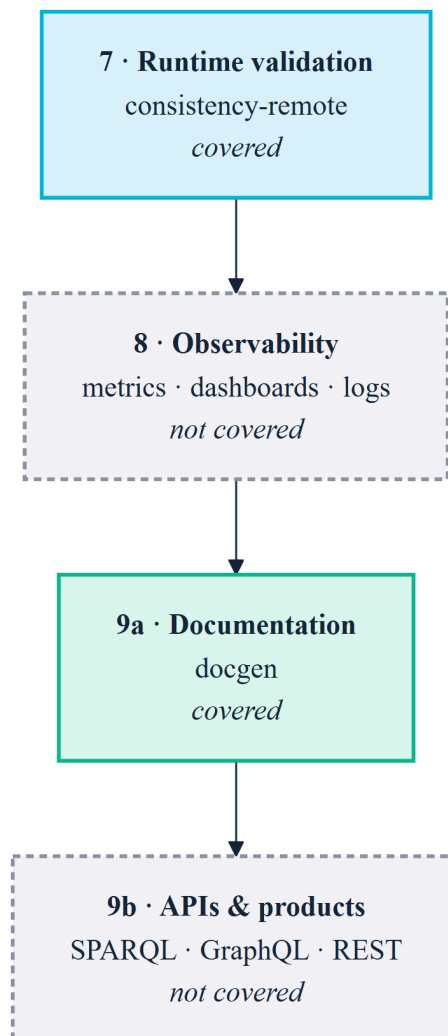
The rest of Level 5 is organisational, and no command produces it.

12. Operate and consume

Part III — The practice · SemOps stages 7–9 · Operating-model layer 6

Stories answered here *As a platform owner, I want the same checks that guard the repository to run against what is actually in the triple store. As an application developer or an agent, I need documentation of what the terms mean and a guarantee they will not change under me.*

The last three pipeline stages — runtime validation, observability, and consumption — are where a knowledge graph either earns its keep or quietly becomes a cost centre. This is also where the toolchain's coverage is thinnest, and this chapter is explicit about which parts are real and which are grey.



12.1 Runtime validation against a live store

Everything so far has run against files in a repository. The moment the org chart moves from Git-tracked Turtle into a real triplestore, a new question appears: **is what is actually loaded still consistent with what we think we shipped?**

Repository checks cannot answer that. A store drifts through partial loads, manual fixes, an update applied to one environment and not another, or a graph someone forgot to reload.

```
python -m ontology_suite consistency-remote \  
--query-endpoint http://localhost:3030/acme/sparql \  
--manifest examples/acme_robotics/manifest.json \  

```

```
--auth-user admin --auth-password secret \
--out-dir out/consistency-remote
```

The manifest is the interesting part

A triplestore holds triples, not project metadata. Fuseki has no built-in link from a named graph back to the transformation query that produced it, nor from a data graph to the ontology graph it is supposed to conform to. The manifest is where a project records that binding explicitly:

```
from ontology_suite.remote.manifest import GraphManifest, GraphBinding

manifest = GraphManifest(bindings=[
    GraphBinding(
        graph_uri="https://acme.example.org/graph/ontology/2.0.0",
        role="ontology",
    ),
    GraphBinding(
        graph_uri="https://acme.example.org/graph/triplified/employees",
        role="triplified_data",
        source_tarql="queries/employees.rq",
        ontology_graph_uri="https://acme.example.org/graph/ontology/2.0.0",
    ),
])
manifest.save("graphs.json")
```

That structure enables a genuine **three-way check**: the live data graph, the ontology graph it claims to conform to, and the local query file that produced it — all compared together. It catches the specific and common failure where the repository is perfectly consistent and the store is a version behind.

--sample-limit caps how many triples are pulled per named graph, which is what makes this viable against a production-sized store.

Named graphs are a decision you make once

Chapter 3 argued that per-partner named graphs are the highest-value cross-boundary decision, and this is where it pays off. The manifest model assumes named graphs; a store where everything was merged into the default graph cannot express these bindings at all, and cannot answer "which supplier asserted this?" either.

It is nearly free at load time and effectively impossible to retrofit.

12.2 Documentation as a product: docgen

SemOps lists documentation as a first-class element — *"without it, semantic systems become opaque"* — and it is the artefact Chapter 2 identified as the answer to the priesthood problem.

```
python -m ontology_suite docgen \
--ontology examples/acme_robotics/acme-org-v1.ttl \
--out-dir out/docgen
```

Real output:

```
Wrote out/docgen/ontology_doc_data.json: prefix='acme', 4 classes,
  2 object properties, 3 datatype properties, 1 sections,
  5 external terms (0 resolved).
Wrote out/docgen/ontology-documentation.html
4 class diagram(s) written to: out/docgen/class-diagrams
```

You get a self-contained ontology-documentation.html — class and property tables, diagrams, and one concise-bounded-description diagram per class — plus ontology_doc_data.json, which is the machine-readable version. That JSON is easy to overlook and is the more strategically useful of the two: it is what you feed a documentation portal, a catalogue, or — per Chapter 5 — an agent that needs to know what your terms mean.

Two rough edges found while running it

--ref **does not sniff serialisation format**. Passing the FOAF vocabulary, which is published as RDF/XML:

```
python -m ontology_suite docgen --ontology acme-org-v1.ttl \
--ref reference_vocab/foaf.rdf --out-dir out/docgen
```

fails with an rdflib Turtle parse error — the RDF/XML comment header is read as Turtle:

```
rdflib.plugins.parsers.notation3.BadSyntax: at line 4 of <>:  
Bad syntax (expected '.' or '}' or ']' at end of statement)
```

Convert the vocabulary to Turtle first, or pass only Turtle files to `--ref`. This is a small thing that costs ten confusing minutes, and it is worth knowing before it happens to you rather than after.

External-term resolution did not engage in this run. Even with `--ref reference_vocab/org.ttl` supplied, the output still reported *"5 external terms (0 resolved)"*. `foaf:Person` and `org:OrganizationalUnit` are correctly *listed* as external terms in both cases — the documentation is accurate about what it does and does not know — but their upstream definitions were not pulled into the page. Reported as observed; the behaviour may be sensitive to how the reference file declares its terms.

Neither undermines the command. The generated page is genuinely the artefact to put in front of a domain expert, and being explicit about unresolved external terms is better than silently omitting them.

Make it automatic

The maturity model expects documentation *"auto-generated on every commit"* at Level 3. That is one CI step:

```
- name: Reference documentation  
  run: |  
    uv run ontology-quality-suite docgen \  
      --ontology ontology/acme-org.ttl --out-dir docs/reference  
- uses: actions/upload-pages-artifact@v3  
  with:  
    path: docs/reference
```

Documentation that regenerates itself is documentation that stays true. Documentation maintained by hand is documentation that was true once.

12.3 Observability: not covered

SemOps stage 8 wants graph size and growth metrics, query-performance dashboards, SPARQL log analysis, data-quality KPIs, provenance tracking and anomaly detection. **None of this is in either tool.** Prometheus, Grafana and Loki are the right answers, exactly as the pipeline blueprint says.

There is one thing worth doing that costs almost nothing, though, and it is the bridge between what you have and what you need:

Every check run writes `full_results.csv`. Retain them, timestamped.

That file is a time series of your semantic quality. Findings-by-severity over time, per check ID, is a genuine data-quality KPI, and it comes free from artefacts CI is already producing. Most teams delete them with the build.

The specific signal to watch is not the absolute count but the **trend in Warnings**. Violations get fixed because they fail the build. Warnings accumulate silently, and an accumulating Warning count is the observable signature of Chapter 1's decay — visible in your own CI artefacts a year before anyone notices the model has drifted.

12.4 Knowledge products and APIs: not covered

SemOps stage 9 wants SPARQL endpoints, GraphQL and REST APIs, search indexes, and data products. Neither tool serves an API, and neither should — that is application development, not quality tooling.

The SemOps material's own microservices guidance is the reference here, and its core architectural point is sound: for low-latency semantic APIs, embed the SPARQL engine **in** the service rather than putting a service in front of a remote triplestore.

Language	Engine	Best at
Rust	Oxigraph	Highest throughput, lowest latency
Python	RDFLib	Analytics, AI/LLM integration

Language	Engine	Best at
Node.js	Comunica	Streaming, web-native, federated
Prolog	SWI semweb	Rule-heavy, expert-system style
Java	Jena / Fuseki	Enterprise-standard, stable

Note that Oxigraph appears twice in this manual's world — as the recommended Rust API engine here, and as the WASM engine the VS Code extension already runs in-process (Chapter 7). That is not a coincidence so much as a signal: the same embeddable engine serves the editor and the endpoint.

What consumption demands from everything upstream

Whether the consumer is a React application, a dashboard, or an agent (Chapter 5), it needs four things — and all four are produced by earlier chapters, not by the API layer:

Consumer needs	Produced by	Chapter
Stable IRIs that do not change meaning	Evidence-based semver, migration annotations	11
To know what a term means	docgen, <code>rdfs:label</code> coverage (QUA-001)	This chapter, 8
Confidence the data conforms	<code>data, consistency-remote</code>	10, this chapter
To know who asserted what	Named graphs	3, §12.1

An API built on a graph that lacks these is an API that will be wrong occasionally and unpredictably — which is worse than being wrong reliably, because consumers stop being able to calibrate their trust.

12.5 Maturity checkpoint

`consistency-remote` against a live store, plus `docgen` on every commit, covers part of Level 4's "*semantic APIs managed and monitored*" — the *managed* half.

The *monitored* half needs an observability stack this toolchain does not provide, and Level 4 also expects Kubernetes deployment, automated environment promotion, and continuous ingestion with retry logic. Those are real gaps, and Chapter 13 is where they are counted honestly rather than glossed.

13. Coverage and gaps

Part IV — The ledger

A manual that only describes what works is marketing. This chapter is the counterweight: **what SemOps asks for, what the toolchain actually delivers, and what it does not.**

Two kinds of entry appear below. *Gaps* are things SemOps requires that these tools do not do at all — usually because they are somebody else's job. *Rough edges* are things the tools do, with behaviour worth knowing before it costs you an afternoon; every one of them was hit while writing this manual.

13.1 Coverage by pipeline stage

Stage	SemOps wants	Covered?	By what
1 · Source control	Artefacts as code, semantic versioning	Partial	version-diff supplies the semver evidence; Git is Git
2 · Modelling & local validation	Model and validate before CI	Yes	VS Code extension; ontology, checks
3 · CI checks	Reject bad semantic changes automatically	Yes	--fail-on, --own-namespace, sketch
4 · Build & packaging	Containers, Helm charts, RDF bundles	No	Docker, Helm — not semantic work
5 · CD to environments	GitOps, environment promotion, IaC	No	Argo CD, Flux, Terraform
6 · Ingestion & ETL	Orchestration, incremental load, provenance	Partial	sketch/triplify/data do transformation and quality; no orchestration
7 · Runtime validation	Enforce constraints against the live store	Yes	consistency-remote
8 · Observability	Metrics, dashboards, log analysis, anomaly detection	No	Prometheus, Grafana, Loki
9 · Consumption	APIs, search, data products	Partial	docgen produces documentation; no API layer

Five of nine substantially covered, two partially, two not at all. The two not covered — packaging/deployment and observability — are the two that are *least* semantic, and have mature general-purpose answers.

13.2 Coverage by operating-model layer

Layer	Covered?	What is real	What is missing
1 · Strategy & Governance	Partial	Evidence for decisions: breaking-change detection, scoped findings, impact analysis	Approval workflows, RBAC, audit trails, stewardship registry
2 · Semantic Development	Yes	Authoring, validation, SHACL, custom checks, docs, versioning	Nothing significant

Layer	Covered?	What is real	What is missing
3 • Semantic CI/CD	Partial	Validation is comprehensive	Packaging and deployment entirely absent
4 • Data Integration	Partial	Transformation, quality checks, conformance	Scheduling, incremental load, retry, streaming, provenance stamping
5 • Platform & Infrastructure	No	—	All of it: K8s, triple-store ops, backup, search index, security
6 • Knowledge Consumption	Partial	Live-store checking, reference documentation	APIs, search, dashboards, self-service

Layer 1 deserves its caveat repeated. No tool approves a change. What this one does is ensure the approver is looking at evidence — *this removes a class, therefore MAJOR; this query breaks; this repair is 100% confident* — rather than at an assertion. The decision stays human, and should.

13.3 Coverage of the ten SemOps elements

#	Element	Covered?	Where
1	Knowledge Lifecycle Management	Yes	Ch. 11
2	Automated Deployment & Packaging	No	—
3	Data Integration & ETL Orchestration	Partial	Ch. 10 — no orchestration
4	Reasoning & Inference Operations	Partial	Reasoning and regression tests yes; incremental reasoning and performance monitoring no
5	Knowledge Graph Observability	No	See §13.5
6	Semantic Governance & Access Control	Partial	Evidence yes; RBAC and workflow no
7	Semantic CI/CD Pipelines	Partial	Linting, SHACL, docs yes; packaging and deployment no
8	Tooling Integration	Partial	Validator, reasoner, CI yes; triple store, K8s, monitoring no
9	Knowledge Products & APIs	No	Ch. 12
10	Documentation & Semantic Literacy	Yes	docgen, generated docs, this manual

13.4 Rough edges, all encountered while writing this manual

Each of these is real, reproducible, and cost time. None is a reason not to use the tools; all are reasons to read this section first.

--own-namespace fails silently on a near-miss

--own-namespace "http://example.org/acme#" against a fixture whose namespace is https://acme.example.org/ns/ returned:

Findings: 0 total (0 Violation, 0 Warning, 0 Info)

It is a **literal IRI-prefix string match**. A wrong scheme, host or trailing separator produces silence, and silence is indistinguishable from success.

Mitigation: copy the namespace from the ontology's @prefix line, and keep one deliberately-failing fixture in CI so a gate that cannot fail is detectable. *Suggested improvement:* warn when --own-namespace matches zero focus nodes while unfiltered findings exist. (Ch. 9)

sketch --queries requires a directory, not a file

FileNotFoundError: No files matching '*.sparql,*.rq,*.tarql,*.tq' found in examples/acme_robotics/employees.rq

The flag reads like it accepts a file. It scans a folder.

Mitigation: pass the containing folder; narrow with --file-pattern. (Ch. 10)

--apply-repairs rewrites comments as well as code

The rename repair is a textual substitution across the whole file. A comment reading "every row is typed acme:Engineer, which v2.0.0 renames to acme:SoftwareEngineer" became "every row is typed acme:SoftwareEngineer, which v2.0.0 renames to acme:SoftwareEngineer" — harmless, and nonsense.

Mitigation: prefer the default dry-run .patch output in automation; review before applying. --apply-repairs suits a developer who will read the diff, not an unattended job that commits its own output. (Ch. 11)

docgen --ref does not sniff serialisation format

Passing FOAF, published as RDF/XML, crashes with an rdflib **Turtle** parse error on the file's XML comment header.

Mitigation: convert to Turtle, or pass only Turtle to --ref. (Ch. 12)

docgen external-term resolution did not engage

Even with --ref reference_vocab/org.ttl, output reported "5 external terms (0 resolved)". Terms are correctly listed as external — the documentation is honest about what it does not know — but upstream definitions were not pulled in. Reported as observed rather than diagnosed. (Ch. 12)

The DL reasoner behaved differently in two commands, same session

The ontology pass emitted REA-022 — *external DL reasoner unavailable*, with an internal UnboundLocalError from the bridge. The data pass, same machine, same fixture, same session, ran HermiT successfully and emitted REA-021, correctly finding acme:Contractor unsatisfiable.

No explanation established. What made it *safe* is that the failing path said so loudly instead of reporting success — the pattern Chapter 4 recommends for every research-lineage dependency.

pySHACL ignores sh:severity inside SPARQL-based constraints

A confirmed upstream bug: severity declared inside sh:sparql constraints is ignored and everything reports as violation. The native Rust engine handles it correctly.

Mitigation: --engine native+sparql where the native engine is available. (Ch. 7)

13.5 The cheapest thing you are not doing

Worth isolating because it is nearly free and almost universally skipped:

Every check run writes full_results.csv. Keep them, timestamped.

That is a time series of semantic quality, produced by CI runs you are already paying for. Findings by severity and by check ID over time is a real data-quality KPI — the closest thing to observability available without a Prometheus stack.

Watch the **Warning trend** specifically. Violations get fixed because they break the build; Warnings accumulate silently, and an accumulating Warning count is the observable signature of decay, visible in your own artefacts a year before anyone notices the model has drifted.

13.6 Gaps that have closed

Worth recording, because the picture is not static. Two limitations documented in an earlier draft of this material no longer hold:

Filtering findings to your own namespace. Previously there was no way to say "only show findings in my own terms" on an import-inclusive run; the recommendation was `--exclude-imports`, which — as Chapter 9 demonstrates with real output — trades 291 irrelevant findings for 4 false ones. `--own-namespace` now does it properly: 5 findings, all genuine, imports still resolved.

Per-row taxonomy values. Previously, a controlled value that varied per data row rather than being hard-coded in the query text was genuinely uncatchable — `pattern-consistency` inspected the query template, and there was no literal to find. Passing `--output-data` now catches it, by comparing values that actually appeared in the produced graph against the declared taxonomy. Verified: the fixture's `MKT` department, absent from a taxonomy declaring only `ENG/QA/ SALES`, is correctly reported (Ch. 11).

The lesson generalises: **re-verify the gaps list against the tools you actually have**. A limitation copied forward from an old document is indistinguishable from a current one, right up until someone wastes a week working around something that was fixed.

13.7 What to do about the real gaps

Gap	Practical answer
Packaging and deployment	Standard container and Helm tooling. A triple store deploys like any stateful service
Orchestration	Airflow, Argo Workflows or Prefect, calling these commands as steps
Observability	Prometheus and Grafana — plus retained <code>full_results.csv</code> as the semantic-quality series
APIs and knowledge products	Application development. Embed the SPARQL engine in the service (Ch. 12)
Approval workflows, RBAC	Branch protection rules and your existing identity platform. The tools supply the evidence; the platform enforces the policy
Incremental and streaming ingestion	Your pipeline's responsibility. These commands are batch
Provenance capture	Named graphs at load time. Decide before your first production load

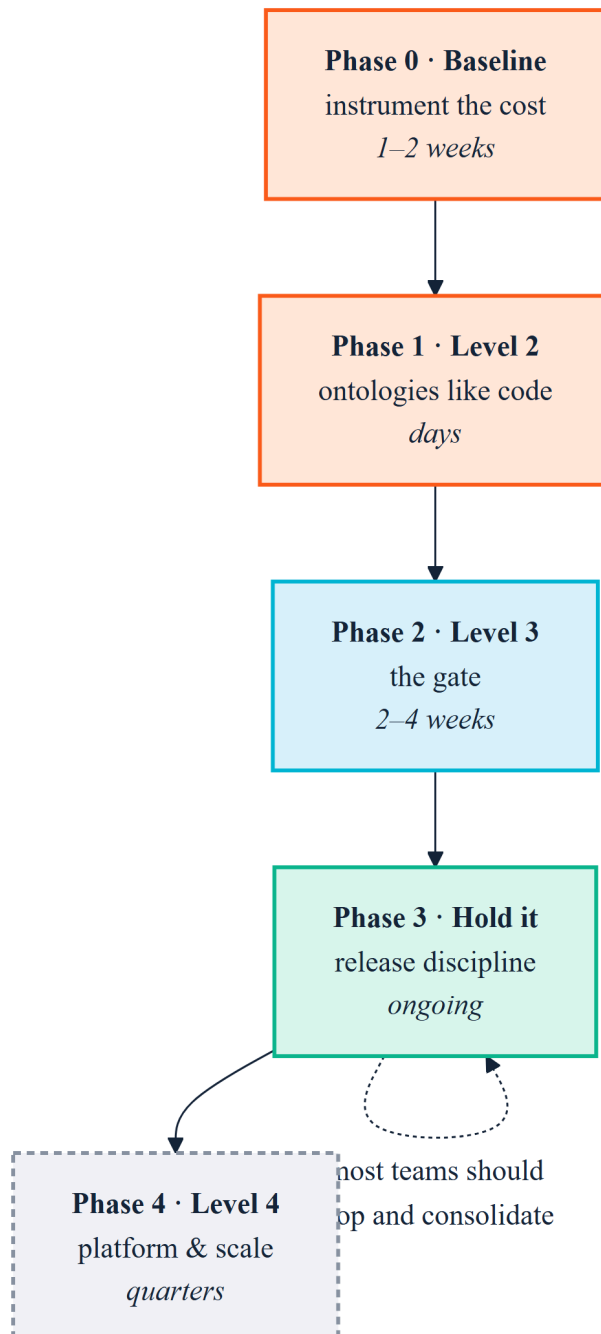
None of these needs a semantic-specific product. That is the useful conclusion: **the parts of SemOps that need specialist tooling are covered, and the parts that are not covered do not need specialist tooling**.

14. Adoption roadmap

Part IV — The ledger

The maturity model tells you where you are. This chapter tells you what to do on Monday.

The sequencing principle throughout: **automate before you accelerate**. Every transition below establishes a control before it increases throughput, for the reason Chapter 5 sets out — an organisation that raises its change rate before it has a gate has industrialised the production of unreviewed model.



Phase 0 — Baseline before you build

One to two weeks. Skipping this is the most common and most expensive mistake.

Once the ontology exists you can no longer measure what it saved, because the counterfactual is gone. Capture the numbers first.

Do this	Why
Ask four analysts to tag two weeks of calendar time as "reconciling definitions"	Produces a cost figure in the business's own terms (Ch. 1)
Record mean lead time for the last five integrations	The curve you are claiming to bend
Time one lineage question end to end	The most legible before/after story you will get
Name the owner	Governance abandonment is the terminal failure mode. If nobody's objectives mention this, you do not have a programme
Pick a first use case with a <i>visible</i> failure	Legible beats valuable. An embarrassing three-week regulator question is worth more as a first case than a larger diffuse inefficiency

Exit criterion: you can state, in one sentence and with a number, what is broken today.

Phase 1 — Reach Level 2: ontologies like code

Days. This is a decision more than a project.

Step	Command or action
Ontology, shapes, queries into Git	—
Install the extension on every modeller's machine	.vsix from the webapp repo
Establish the local loop	<i>Run Local Checks, Ontology Outline, Show Metrics</i>
Run the tightest gate by hand	<code>ontology --ontology <yours> --import-dir vendor/vocab</code>
Vendor your imports	Copy upstream vocabularies into vendor/vocab; never rely on <code>--allow-network</code>
Record your namespace	Copy the exact <code>@prefix</code> string somewhere the team can find. You will need it repeatedly, and getting it wrong fails silently (Ch. 9)

Two habits to establish now, while it is cheap:

Design the IRI scheme deliberately. Schema terms and instance data should sit under a common prefix you can filter on — `https://acme.example.org/ns/` and `https://acme.example.org/data/` share `https://acme.example.org/`, which is what makes `--own-namespace` usable across both (Ch. 10). Retrofitting an IRI scheme is a migration.

Decide named graphs before the first production load. Per-source named graphs are nearly free at load time and effectively impossible to retrofit (Ch. 12).

Exit criterion: every modeller sees findings in their editor, and the ontology's history is in Git.

Phase 2 — Reach Level 3: the gate

Two to four weeks. This is the transition worth real effort.

It converts "we have standards" into "the standards are enforced without anyone remembering to enforce them" — and it is the prerequisite for everything in Phases 3 and 4.

Week 1 — the gate that cannot be ignored

```
ontology-quality-suite ontology \  
  --ontology ontology/acme-org.ttl --import-dir vendor/vocab \  
  --fail-on Violation
```

Start here, not with the full registry. It is the narrowest, highest-signal check, and a gate people trust from day one is worth more than a comprehensive one they learn to bypass.

Week 2 — scope the registry, then add it

```
ontology-quality-suite checks \  
  --ontology ontology/acme-org.ttl --import-dir vendor/vocab \  
  --own-namespace "https://acme.example.org/" \  
  --engine sparql --fail-on Violation
```

Do not skip `--own-namespace`. Unscoped, the fixture in this manual produces 296 findings where 5 are yours; the gate is dead within two sprints (Ch. 9). And do not reach for `--exclude-imports` instead — verified, it introduces four *false* findings, which damages trust faster than noise does.

Verify the filter is live by pointing it at a known-failing input once. A gate that cannot fail is indistinguishable from a gate that passes.

Week 3 — the pipeline stages

PR touches	Gate
A transformation query	sketch — needs no CSV, runs in seconds
Data	data --own-namespace, plus --sample N if large
A taxonomy	pattern-consistency --output-data

`--output-data` is not optional in practice. Without it, per-row controlled values are invisible (Ch. 11).

Week 4 — documentation and artefacts

```
ontology-quality-suite docgen --ontology ontology/acme-org.ttl \  
  --out-dir docs/reference
```

Publish it. Then **retain** `full_results.csv` from every run, timestamped — it costs nothing and becomes your only quality time series (Ch. 13).

Exit criterion: a pull request that breaks the ontology fails, and the failure names something the team owns.

Phase 3 — Hold it: release discipline

Ongoing. Most teams should stop and consolidate here.

Practice	Command	Cadence
Evidence-based semver	version-diff old.ttl new.ttl --json	Every release
Change impact	consistency --new ... --old ... --queries ...	Every breaking change
Migration annotations	oldTerm owl:equivalentClass newTerm — from the retiring IRI	Every rename
Wide audit	checks with imports, unscoped, --fail-on never	Weekly, or on upstream change
Live-store check	consistency-remote --manifest graphs.json	After every deployment

Add one governance control, which is a branch protection rule rather than a command: **a MAJOR bump requires a named reviewer**. The tool establishes the fact; the platform enforces the policy (Ch. 11).

And one review-checklist item that will otherwise cost you an afternoon per occurrence: migration annotations are read **directionally**, from the retiring IRI to the replacement. Written the other way round, rename detection silently degrades to name-similarity guessing.

If you work across organisational boundaries

Three additions, all cheap (Ch. 3):

- **Publish your check registry** as a directory partners can point `--registry/--shapes/--sparql` at, so they validate before submitting rather than after rejection.
- **Accept CSV**. The triplify path means a partner never authors RDF.
- **Attach diff.json** to every change notice, so partner tooling can act without parsing your prose.

Phase 4 — Level 4, and knowing when not to Quarters, and mostly not semantic work.

Level 4 is Kubernetes deployment, environment promotion, continuous ingestion with retry logic, an observability stack, and managed semantic APIs. Almost none of it is in this toolchain, and almost none of it needs specialist semantic tooling (Ch. 13).

What is here for Level 4:

Need	Command
Speed at scale	<code>--engine native+sparql</code>
Large data graphs	<code>data --sample N</code>
Live-store validation	<code>consistency-remote --sample-limit N</code>

Consider not doing Phase 4. A team solidly at Level 3, with a gate that holds and releases that are evidence-based, has captured most of the available value. Level 4 is justified by *scale* — many consumers, large graphs, strict availability. Pursued for its own sake it is an expensive way to acquire operational burden.

And Level 5 is overwhelmingly organisational. A team frustrated that tooling will not carry them there has correctly identified that the remaining problem is Chapter 2's, not the toolchain's.

The first ninety days, on one page

Days	Focus	Deliverable
1–10	Baseline	A number, a named owner, one legible use case
11–20	Level 2	Ontology in Git; extension on every desk; IRI scheme decided
21–40	The gate	<code>ontology --fail-on Violation</code> in CI, and it has failed at least once
41–60	Scope and extend	<code>checks --own-namespace</code> ; sketch on query PRs; first house rule written
61–75	Release discipline	<code>version-diff</code> on release; MAJOR requires a reviewer
76–90	Make it visible	<code>docgen</code> published; <code>full_results.csv</code> retained; report the baseline delta

The last row matters more than it looks. A programme that reaches day 90 without having reported back against Phase 0's number will be asked to justify itself in a language it did not prepare for.

The shortest version

If you read nothing else in this manual:

1. **Measure the cost before you fix it.** You cannot reconstruct the counterfactual.
2. **Name an owner.** Unowned ontologies decay silently, and decay is the dominant failure mode.
3. **Gate on `--fail-on` Violation, scoped with `--own-namespace`.** Unscoped gates die; `--exclude-imports` produces false findings.
4. **Assert migration annotations from the retiring IRI.** Symmetric in logic, directional in tooling.
5. **Automate before you accelerate.** Especially before adding agents.

Appendix A — Diagram and brand conventions

Every diagram in this manual is Mermaid, rendered natively by GitHub, and is built to two hard constraints:

1. **Screen** — readable at GitHub's rendered document width without zooming.
2. **A4 print** — readable in portrait A4 (≈ 170 mm of usable width) after the browser scales the page to fit.

The second constraint is the binding one. A diagram that is comfortable on a 27-inch monitor routinely becomes unreadable at A4 because the whole SVG is scaled down uniformly to fit the page width — halve the width and you halve the type size. Everything below exists to stop that happening.

A.1 The width rule

A diagram may be at most 5 nodes wide. Beyond that, go vertical.

Mermaid lays a flowchart out along its declared direction and grows the SVG until every node fits. There is no wrapping. A ten-stage `flowchart LR` becomes a very wide, very short SVG; scaled into an A4 column it renders at roughly 7-point type and is functionally useless on paper.

The rule in practice:

Nodes in the longest rank	Direction to use
2–4	<code>flowchart LR</code> is fine, and reads well
5	<code>LR</code> only if labels are short; otherwise <code>TD</code>
6+	<code>flowchart TD</code> , or split into two diagrams

The nine-stage SemOps pipeline is never drawn as one nine-wide row anywhere in this manual. It is drawn top-down, or split into three groups of three. That is a deliberate consequence of this rule, not an accident of taste.

A.2 The label rule

At most ~28 characters per label line. Break with `<br/>`, never rely on Mermaid to wrap — it does not.

A single long label sets the width of its entire rank and drags the whole diagram wider. Prefer:

```
A["Ontology PR gate<br/>checks --own-namespace"]
```

over:

```
A["Ontology PR gate running checks with the own-namespace filter"]
```

Subtitles and qualifiers belong in the prose under the diagram, not inside the node. A diagram that needs a paragraph inside a box is a table in disguise.

A.3 The size rule

Aim for 12 nodes. Hard-stop at 16.

Past roughly sixteen boxes a reader stops tracing edges and starts scanning for the one box they care about — at which point a table serves them better and prints perfectly. Several places in this manual deliberately use a table where a diagram would have been the flashier choice, for exactly this reason. See §A.7.

A.4 Colour: explicit fills, always

GitHub renders Mermaid against both a light and a dark page theme, and readers print from either. A node with no explicit fill inherits a theme-dependent background; a node with an explicit fill but no explicit text colour can end up dark-on-dark.

Every styled node sets `fill`, `stroke`, and `color` together.

Light fills with the dark navy text colour survive both themes, and print legibly on a monochrome laser printer, where the palette collapses to greys of visibly different densities.

The palette

Sampled from the Semantechs mark in `assets/`:

Role in a diagram	Stroke	Fill	Sampled from
Authoring / human work	#FD5B1C	#FFE6DA	turtle shell, upper field
Automation / CI gates	#02B7D4	#D9F4FA	the SPARQL hook
Data and ingestion	#7CBA07	#ECF7D5	turtle body
Operations / runtime	#0CB88E	#D9F5EC	lower-right field
Governance / decision	#FE8902	#FFF0D6	owl plumage
Not covered by tooling	#8A93A0	#F1F3F5	— (deliberately grey)
Text and edges	#14243A	—	outline ink

Grey is reserved. It means "SemOps asks for this and the toolchain in this manual does not provide it" — the visual counterpart of Chapter 13. Never use grey decoratively.

The standard preamble

Copy this block verbatim into new diagrams:

```
%%{init: {'theme':'base','themeVariables':{
  'fontSize':'15px','fontFamily':'Segoe UI, Helvetica, Arial, sans-serif',
  'lineColor':'#14243A','primaryTextColor':'#14243A',
  'edgeLabelBackground':'#FFFFFF'}}}%%
```

`fontSize` is set at 15px rather than the 16px default because it holds a little more text per rank at the same rendered width — which, per §A.1, is the scarce resource. Do not go below 14px; the A4 downscale is unforgiving.

Then declare only the classes the diagram actually uses:

```
classDef human fill:#FFE6DA,stroke:#FD5B1C,stroke-width:2px,color:#14243A;
classDef ci fill:#D9F4FA,stroke:#02B7D4,stroke-width:2px,color:#14243A;
classDef data fill:#ECF7D5,stroke:#7CBA07,stroke-width:2px,color:#14243A;
classDef ops fill:#D9F5EC,stroke:#0CB88E,stroke-width:2px,color:#14243A;
classDef govern fill:#FFF0D6,stroke:#FE8902,stroke-width:2px,color:#14243A;
classDef gap fill:#F1F3F5,stroke:#8A93A0,stroke-width:2px,color:#14243A,stroke-dasharray:4 3;
```

The dashed border on `gap` carries the same meaning as the grey fill, so the distinction survives both colour-blind readers and monochrome printing. No meaning in this manual is ever carried by hue alone.

A.5 Checking a diagram before committing

Three checks, in increasing order of effort:

1. **Count.** Longest rank ≤ 5 , longest label line ≤ 28 characters, nodes ≤ 16 .
2. **Screen.** Push to a branch and view the rendered Markdown on GitHub. Mermaid in a local editor preview does not always agree with GitHub's renderer.
3. **Print.** `Ctrl+P` from the GitHub page, A4 portrait, "Fit to page width", and read the smallest label. If you are squinting, apply §A.1 and split it.

Step 3 is the one people skip and the one that catches real problems.

A.6 Accessibility

- Alt text on every raster image.
- No meaning carried by colour alone — pair every colour distinction with position, a border style, or a word in the label.
- Palette contrast: all six fills carry the #14243A text colour at a contrast ratio above 12:1, comfortably clear of the WCAG AA 4.5:1 threshold for body text.

A.7 When not to draw a diagram

A diagram earns its place when it shows a **mechanism** — a flow, a gate, a loop, a boundary crossing. It does not earn its place when it is:

- an unordered list of things (use a list);
- a mapping from one set to another (use a table — it also prints perfectly and is searchable, which an SVG is not);
- a sequence of commands (use a fenced code block).

Chapter 13's coverage matrix is the clearest case: six operating-model layers against two toolchains is a table, and would have been a worse diagram.

A.8 The mark

The Semantechs mark — turtle, owl, hook, XML — reads as a summary of the stack this manual operates: **Turtle** the serialisation, **OWL** the ontology language, the hook for **SPARQL**, and XML for the RDF/XML lineage that most real-world vocabularies (FOAF among them, as Chapter 8 shows) are still published in.

Full-resolution original: `assets/semantechs-logo.png` (1024×1024). Use `assets/semantechs-logo-320.png` for in-page headers — at 320 px it is a tenth of the file size and no page in this manual displays the mark above 160 px.

A.9 The print edition

`tools/build-pdf.mjs` produces the A4 PDF. It renders every Mermaid block to a PNG through a headless Chromium, assembles the chapters into one Markdown file, generates the index, and runs `pandoc → typst`.

```
cd tools && npm install && npx puppeteer browsers install chrome-headless-shell
node build-pdf.mjs
```

Four things learned building it are worth knowing before you add a diagram.

Mermaid needs a real browser. Its layout comes from actual text metrics (`getBBBox`), which no DOM shim supplies accurately. A `jsdom`-based render produces boxes of the wrong size. Diagrams are rasterised at `deviceScaleFactor: 3` rather than passed through as SVG, so their text never depends on the PDF engine resolving the right webfont.

The width rule has a hard number in print. A diagram is placed into the ~170 mm text block, so a diagram W pixels wide renders its 15px labels at $15 / W \times 170$ mm. Keeping labels at or above 8pt means:

$W \leq \sim 910$ px. The build prints a warning for any diagram over that.

Two diagrams originally breached it — both because side-by-side subgraphs, not node count, set the width. §A.1's "5 nodes per rank" rule does not constrain subgraphs, so check the rendered pixel width, not just the node count.

Force subgraphs to stack with invisible links. Mermaid places unconnected subgraphs side by side, which is what blew the width budget. An invisible link stacks them vertically and improves the web rendering too:

```
PEER ~~~ CHAIN ~~~ OPEN
```

Beware cycles. A feedback edge drawn straight back to the first node makes Mermaid's layout break the cycle by hoisting the *last* group to the top — the nine-stage pipeline rendered as Run/Author/Ship until the loop was redrawn as a terminal node. If a diagram's vertical order looks wrong, a back edge is why.

Index

Entries reference **chapters**, not pages — they are resolved from the source text, so they stay correct as the manual is revised. Use the outline at the front for page numbers.

Commands

`ontology-quality-suite <command>`, or `python -m ontology_suite <command>`.

Entry	Chapters
checks	8, 9, 14
consistency	11, 12
consistency-remote	6, 7, 12, 13
data	10
docgen	2, 6, 7, 12, 13, 14
ontology	8, 9, 14
pattern-consistency	7, 9, 11, 13
run	9
sketch	6, 7, 9, 10, 13, 14
triplify	6, 7, 10, 13
version-diff	1, 3, 5, 6, 7, 9, 11, 13, 14

Flags and options

Entry	Chapters
<code>--allow-network</code>	9, 14
<code>--apply-repairs</code>	6, 11, 13
<code>--auth-password</code>	12
<code>--auth-user</code>	12
<code>--csv-dir</code>	10
<code>--data</code>	8
<code>--dedup</code>	10
<code>--delimiter</code>	10
<code>--engine</code>	6, 7, 8, 9, 10, 13, 14
<code>--exclude-imports</code>	9, 11, 13, 14
<code>--extra</code>	4, 7
<code>--fail-on</code>	1, 6, 8, 9, 10, 11, 13, 14
<code>--fail-on-misalignment</code>	9
<code>--fail-on-mismatch</code>	9
<code>--file-pattern</code>	10, 13
<code>--import-dir</code>	8, 9, 10, 11, 14
<code>--json</code>	3, 9, 11, 14
<code>--manifest</code>	12, 14
<code>--min-confidence</code>	6, 11

Entry	Chapters
--new	11, 14
--no-header-row	10
--ntriples	10
--old	11, 14
--ontology	8, 9, 10, 11, 12, 14
--out-dir	8, 9, 10, 11, 12, 14
--output-data	11, 13, 14
--own-namespace	6, 7, 9, 10, 11, 13, 14, A
--oxi-gen-bin	10
--profile	8
--queries	9, 10, 11, 13, 14
--query-endpoint	12
--ref	12, 13
--registry	3, 6, 8, 9, 14
--sample	6, 9, 10, 14
--sample-limit	12, 14
--shapes	3, 6, 8, 9, 14
--sparql	3, 6, 8, 9, 14
--tab	10
--taxonomy	11
--test	10
--verbose	7, 9

Check identifiers

Entry	Chapters
ACM-001	8
CNF-005	10
DAT-002	9, 10
LOG-001	2, 4, 5, 7, 8, 9, 10
QUA-001	5, 7, 9, 10, 12
QUA-004	9
REA-001	4
REA-021	10, 13
REA-022	4, 7, 8, 10, 13
STR-003	7, 9, 10
STR-006	10
STY-002	7, 9, 10
VOC-001	8

VS Code commands

Command	Chapters
Add Class	7
Add Sibling Class	7, 8
Add Subclass	7, 8
Infer Ontology + Query from CSV	7, 10
New Ontology	7
Query Workbench	7, 10
Run Deep Validation	7
Run Full Triplify	7
Run Local Checks	7, 8, 14
Show Metrics & DL Expressivity	7, 8
Visualize Subject Graph	7, 8